

STAR RX72N - gpio_test Bench Test
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 gpio_test Directory Reference	5
4 File Documentation	7
4.1 gpio_test/clock.c File Reference	7
4.1.1 Detailed Description	8
4.1.2 Enumeration Type Documentation	8
4.1.2.1 clock_byte_constants_t	8
4.1.2.2 clock_constants_word_t	9
4.1.2.3 clock_extra_addrs_t	9
4.1.2.4 clock_sckcr_t	9
4.1.3 Function Documentation	10
4.1.3.1 clock_init()	10
4.1.3.2 packcr_reg()	11
4.2 clock.c	11
4.3 gpio_test/cmt0.c File Reference	13
4.3.1 Detailed Description	14
4.3.2 Enumeration Type Documentation	15
4.3.2.1 cmt0_cfg_t	15
4.3.2.2 cmt_addr_t	15
4.3.2.3 icu_cmt0_cfg_t	16
4.3.3 Function Documentation	16
4.3.3.1 _tx_timer_interrupt()	16
4.3.3.2 cmt0_init()	16
4.3.3.3 cmt0_isr()	18
4.3.3.4 internal_cmstr0()	19
4.3.3.5 internal_cmt0_cmcent()	20
4.3.3.6 internal_cmt0_cmcor()	20
4.3.3.7 internal_cmt0_cmcr()	20
4.3.3.8 internal_icu_ier()	21
4.3.3.9 internal_icu_ipr()	21
4.3.3.10 internal_icu_ir()	22
4.3.3.11 internal_mstpcra()	22
4.3.3.12 internal_prcr()	23
4.4 cmt0.c	23
4.5 gpio_test/linker.ld File Reference	25
4.6 linker.ld	25
4.7 gpio_test/main.c File Reference	27

4.7.1 Detailed Description	28
4.7.2 Data Structure Documentation	29
4.7.2.1 struct pin_desc_t	29
4.7.3 Enumeration Type Documentation	29
4.7.3.1 port_bit_t	29
4.7.3.2 port_offset_t	30
4.7.3.3 port_reg_base_t	31
4.7.3.4 rtos_prio_t	31
4.7.3.5 rtos_stack_t	32
4.7.3.6 timing_t	32
4.7.4 Function Documentation	32
4.7.4.1 clock_init()	32
4.7.4.2 cmt0_init()	33
4.7.4.3 internal_delay()	35
4.7.4.4 internal_gpio_init_all()	36
4.7.4.5 internal_pdr()	37
4.7.4.6 internal_pmr()	38
4.7.4.7 internal_podr()	38
4.7.4.8 internal_sweep_task_entry()	39
4.7.4.9 main()	40
4.7.4.10 tx_application_define()	41
4.7.5 Variable Documentation	42
4.7.5.1 s_pins	42
4.7.5.2 s_sweep_stack	43
4.7.5.3 s_sweep_tcb	43
4.8 main.c	43

Chapter 1

Directory Hierarchy

1.1 Directories

gpio_test	5
clock.c	7
cmt0.c	13
linker.ld	25
main.c	27

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

gpio_test/ clock.c	
RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48 . . .	7
gpio_test/ cmt0.c	
CMT0 100 Hz tick source for ThreadX with PCKB = 48 MHz	13
gpio_test/ linker.ld	25
gpio_test/ main.c	
GPIO breakout verification firmware for RX72N, ThreadX edition	27

Chapter 3

Directory Documentation

3.1 gpio_test Directory Reference

Files

- file [clock.c](#)
RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.
- file [cmt0.c](#)
CMT0 100 Hz tick source for ThreadX with PCKB = 48 MHz.
- file [linker.ld](#)
- file [main.c](#)
GPIO breakout verification firmware for RX72N, ThreadX edition.

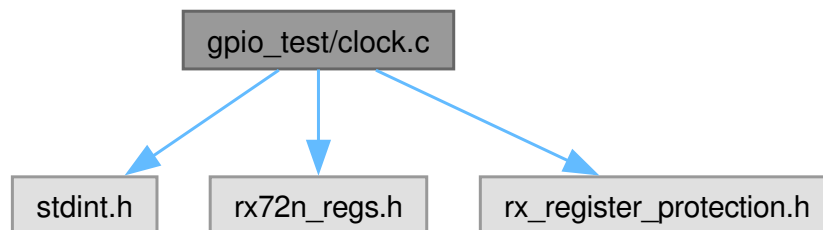
Chapter 4

File Documentation

4.1 gpio_test/clock.c File Reference

RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.

```
#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
Include dependency graph for clock.c:
```



Enumerations

- enum `clock_extra_addrs_t` : `uintptr_t` { `k_packcr_addr` = 0x00080044U }
- enum `clock_constants_word_t` : `uint16_t` { `k_pllcr_hoco_x12` = 0x1710U , `k_sckcr2_uck_div4` = 0x0031U , `k_packcr_pll_source` = 0x0000U }
- enum `clock_sckcr_t` : `uint32_t` { `k_sckcr_value` = 0x21C21211U }
- enum `clock_byte_constants_t` : `uint8_t` {
 `k_hococr_running` = 0x00U , `k_pllcr2_enable` = 0x00U , `k_memwait_one` = 0x01U ,
 `k_oscovfsr_hcovf` = 0x08U ,
 `k_oscovfsr_plovf` = 0x04U }

Functions

- static volatile `uint16_t` * `packcr_reg` (void)
- void `clock_init` (void)

4.1.1 Detailed Description

RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.

This test sources the PLL from the internal HOCO (16 MHz) instead of the STAR PCB's 24 MHz external crystal on MOSC/EXTAL. The choice is deliberate (isolate the PLL bring-up from crystal startup), not because the board lacks a crystal – it has one.

Target clocks:

- PLL = HOCO 16 MHz x 12 = 192 MHz
- ICLK = 192 / 2 = 96 MHz (CPU)
- PCKA = 192 / 2 = 96 MHz
- PCKB = 192 / 4 = 48 MHz (CMT0 ThreadX tick)
- PCKC = 192 / 2 = 96 MHz
- PCKD = 192 / 2 = 96 MHz
- FCLK = 192 / 4 = 48 MHz (flash)
- BCLK = 192 / 4 = 48 MHz
- UCLK = 192 / 4 = 48 MHz (not used, but set for completeness)

Path: HOCO 16 MHz -> PLIDIV /1 -> STC x12 -> 192 MHz PLL output.

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [clock.c](#).

4.1.2 Enumeration Type Documentation

4.1.2.1 clock_byte_constants_t

enum [clock_byte_constants_t](#) : uint8_t

Enumerator

k_hococr_running	HOCOCR: 0 = HOCO running
k_pllcr2_enable	PLLCR2: 0 = PLL running (inverted logic)
k_memwait_one	MEMWAIT: 1 wait state for ICLK > 120 MHz
k_oscovfsr_hcovf	OSCOVFSR bit 3: HOCO stable
k_oscovfsr_plovf	OSCOVFSR bit 2: PLL stable

Definition at line 75 of file [clock.c](#).

```
00075      : uint8_t {
00076  k_hococr_running = 0x00U, /**< HOCOCR: 0 = HOCO running */
00077  k_pllcr2_enable  = 0x00U, /**< PLLCR2: 0 = PLL running (inverted logic) */
00078  k_memwait_one    = 0x01U, /**< MEMWAIT: 1 wait state for ICLK > 120 MHz */
00079  k_oscovfsr_hcovf = 0x08U, /**< OSCOVFSR bit 3: HOCO stable */
00080  k_oscovfsr_plovf = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00081 } clock_byte_constants_t;
```

4.1.2.2 clock_constants_word_t

enum [clock_constants_word_t](#) : uint16_t

Enumerator

k_pllcr_hoco_x12	
k_sckcr2_uck_div4	
k_packcr_pll_source	

Definition at line 43 of file [clock.c](#).

```

00043         : uint16_t {
00044     /*
00045      * PLLCR layout (16 bits):
00046      * bits [1:0] PLIDIV : 00=/1, 01=/2, 10=/3
00047      * bit [4] PLLSRCSEL: 0=MOSC, 1=HOCO
00048      * bits [13:8] STC : multiply = (STC + 1) / 2
00049      *
00050      * HOCO 16 MHz / 1 * 12 = 192 MHz PLL output.
00051      * STC = (12 * 2) - 1 = 23 = 0x17.
00052      * PLLSRCSEL = 1 (HOCO) -> bit 4 set.
00053      */
00054     k_pllcr_hoco_x12 = 0x1710U,
00055
00056     /*
00057      * SCKCR2: bits [7:4] UCK = /(N+1), bits [3:0] reserved (must be 0x1).
00058      * UCK = 0x3 -> /4 -> 192 MHz / 4 = 48 MHz exact.
00059      */
00060     k_sckcr2_uck_div4 = 0x0031U,
00061
00062     /* PACKCR: bit 0 UPLLSEL -- 0 routes UCLK divider input from main PLL. */
00063     k_packcr_pll_source = 0x0000U,
00064 } clock_constants_word_t;

```

4.1.2.3 clock_extra_addrs_t

enum [clock_extra_addrs_t](#) : uintptr_t

Enumerator

k_packcr_addr	Peripheral clock source: 0=PLL, 1=PPLL
---------------	--

Definition at line 36 of file [clock.c](#).

```

00036         : uintptr_t {
00037     k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL, 1=PPLL */
00038 } clock_extra_addrs_t;

```

4.1.2.4 clock_sckcr_t

enum [clock_sckcr_t](#) : uint32_t

Enumerator

k_sckcr_value	
---------------	--

Definition at line 66 of file [clock.c](#).

```

00066         : uint32_t {
00067     /*
00068      * SCKCR dividers:
00069      * FCK=/4 (48), ICK=/2 (96), PSTOP1=1, PSTOP0=1, BCK=/4 (48),
00070      * PCKA=/2 (96), PCKB=/4 (48), PCKC=/2 (96), PCKD=/2 (96).
00071      */
00072     k_sckcr_value = 0x21C21211U,
00073 } clock_sckcr_t;

```

4.1.3 Function Documentation

4.1.3.1 clock_init()

```
void clock_init (
    void )
```

HOCO 16 MHz -> PLL 192 MHz -> ICLK 96 MHz

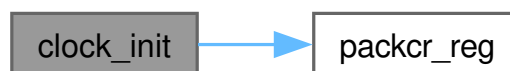
Definition at line 100 of file [clock.c](#).

```
00101 {
00102     volatile rx_system_regs_t *sys = system_regs();
00103
00104     /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */
00105     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00106
00107     /* Step 1: start HOCO if not already running. */
00108     sys->hococr = k_hococr_running;
00109
00110     /* Step 2: wait for HOCO to stabilise. */
00111     while ((sys->oscovfsr & k_oscovfsr_hcovf) == 0U) {
00112         __asm__ volatile("nop");
00113     }
00114
00115     /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */
00116     sys->pllcr = k_pllcr_hoco_x12;
00117     sys->pllcr2 = k_pllcr2_enable;
00118
00119     /* Step 4: wait for PLL to stabilise. */
00120     while ((sys->oscovfsr & k_oscovfsr_plovf) == 0U) {
00121         __asm__ volatile("nop");
00122     }
00123
00124     /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */
00125     *memwait_reg() = k_memwait_one;
00126
00127     /* Step 6: program all bus dividers. */
00128     sys->sckcr = k_sckcr_value;
00129
00130     /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131     *packcr_reg() = k_packcr_pll_source;
00132
00133     /* Step 8: program the USB clock divider. */
00134     sys->sckcr2 = k_sckcr2_uck_div4;
00135
00136     /* Step 9: switch the system clock source to PLL. */
00137     sys->sckcr3 = k_sckcr3_cksel_pll;
00138
00139     /* Step 10: relock PRCR. */
00140     *prcr_reg() = k_rx_prcr_lock;
00141 }
```

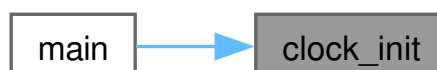
References [k_hococr_running](#), [k_memwait_one](#), [k_oscovfsr_hcovf](#), [k_oscovfsr_plovf](#), [k_packcr_pll_source](#), [k_pllcr2_enable](#), [k_pllcr_hoco_x12](#), [k_sckcr2_uck_div4](#), [k_sckcr_value](#), and [packcr_reg\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.3.2 packcr_reg()

```
volatile uint16_t * packcr_reg (
    void ) [inline], [static]
```

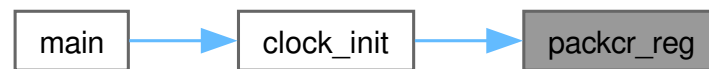
Definition at line 86 of file [clock.c](#).

```
00087 {
00088     return (volatile uint16_t *)k_packcr_addr;
00089 }
```

References [k_packcr_addr](#).

Referenced by [clock_init\(\)](#).

Here is the caller graph for this function:



4.2 clock.c

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file clock.c
00003  * @brief RX72N clock initialisation: HOCO 16 MHz -> PLL 192 -> ICLK 96 / PCKB 48.
00004  *
00005  * @details
00006  * This test sources the PLL from the internal HOCO (16 MHz) instead of
00007  * the STAR PCB's 24 MHz external crystal on MOSC/EXTAL. The choice is
00008  * deliberate (isolate the PLL bring-up from crystal startup), not because
00009  * the board lacks a crystal -- it has one.
00010  *
00011  * Target clocks:
00012  * - PLL = HOCO 16 MHz x 12 = 192 MHz
00013  * - ICLK = 192 / 2 = 96 MHz (CPU)
00014  * - PCKA = 192 / 2 = 96 MHz
00015  * - PCKB = 192 / 4 = 48 MHz (CMT0 ThreadX tick)
00016  * - PCKC = 192 / 2 = 96 MHz
00017  * - PCKD = 192 / 2 = 96 MHz
00018  * - FCLK = 192 / 4 = 48 MHz (flash)
00019  * - BCLK = 192 / 4 = 48 MHz
00020  * - UCLK = 192 / 4 = 48 MHz (not used, but set for completeness)
00021  *
00022  * Path: HOCO 16 MHz -> PLIDIV /1 -> STC x12 -> 192 MHz PLL output.
00023  *
00024  * SPDX-License-Identifier: MIT
00025  * @copyright Copyright (c) 2026 Locked Inc.
00026  */
00027
00028 #include <stdint.h>
00029
00030 #include "rx72n_regs.h"
00031 #include "rx_register_protection.h"
00032
00033 /**
00034  * Register addresses not exposed by the rx_system_regs_t struct
00035  */
00036 typedef enum : uintptr_t {
00037     k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL, 1=PPLL */
00038 } clock_extra_addrs_t;
00039
00040 /**
00041  * Clock configuration constants
00042  */
00043 typedef enum : uint16_t {
```

```

00044  /*
00045  * PLLCR layout (16 bits):
00046  *   bits [1:0] PLIDIV : 00=/1, 01=/2, 10=/3
00047  *   bit  [4]  PLLSRCSEL: 0=MOSC, 1=HOCO
00048  *   bits [13:8] STC    : multiply = (STC + 1) / 2
00049  *
00050  * HOCO 16 MHz / 1 * 12 = 192 MHz PLL output.
00051  * STC = (12 * 2) - 1 = 23 = 0x17.
00052  * PLLSRCSEL = 1 (HOCO) -> bit 4 set.
00053  */
00054  k_pllcr_hoco_x12 = 0x1710U,
00055
00056  /*
00057  * SCKCR2: bits [7:4] UCK = /(N+1), bits [3:0] reserved (must be 0x1).
00058  * UCK = 0x3 -> /4 -> 192 MHz / 4 = 48 MHz exact.
00059  */
00060  k_sckcr2_uck_div4 = 0x0031U,
00061
00062  /* PACKCR: bit 0 UPLLSEL -- 0 routes UCLK divider input from main PLL. */
00063  k_packcr_pll_source = 0x0000U,
00064 } clock_constants_word_t;
00065
00066 typedef enum : uint32_t {
00067     /*
00068     * SCKCR dividers:
00069     *   FCK=/4 (48), ICK=/2 (96), PSTOP1=1, PSTOP0=1, BCK=/4 (48),
00070     *   PCKA=/2 (96), PCKB=/4 (48), PCKC=/2 (96), PCKD=/2 (96).
00071     */
00072     k_sckcr_value = 0x21C21211U,
00073 } clock_sckcr_t;
00074
00075 typedef enum : uint8_t {
00076     k_hococr_running = 0x00U, /**< HOCOCR: 0 = HOCO running */
00077     k_pllcr2_enable = 0x00U, /**< PLLCR2: 0 = PLL running (inverted logic) */
00078     k_memwait_one = 0x01U, /**< MEMWAIT: 1 wait state for ICLK > 120 MHz */
00079     k_oscovfsr_hcovf = 0x08U, /**< OSCOVFSR bit 3: HOCO stable */
00080     k_oscovfsr_plovf = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00081 } clock_byte_constants_t;
00082
00083 /*
=====
00084 * Inline accessors
00085 *
=====
*/
00086 static inline volatile uint16_t *packcr_reg(void)
00087 {
00088     return (volatile uint16_t *)k_packcr_addr;
00089 }
00090
00091 /*
=====
00092 * clock_init -- bring the chip from LOCO 240 kHz to PLL 192/96/48 MHz.
00093 *
00094 * Uses HOCO (16 MHz internal oscillator) as the PLL source.
00095 * No external crystal required.
00096 *
00097 * Must run before any peripheral that depends on PCKB is touched.
00098 * Called from main() before cmt0_init() and before tx_kernel_enter().
00099 *
=====
*/
00100 void clock_init(void)
00101 {
00102     volatile rx_system_regs_t *sys = system_regs();
00103
00104     /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */
00105     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00106
00107     /* Step 1: start HOCO if not already running. */
00108     sys->hococr = k_hococr_running;
00109
00110     /* Step 2: wait for HOCO to stabilise. */
00111     while ((sys->oscovfsr & k_oscovfsr_hcovf) == 0U) {
00112         __asm__ volatile("nop");
00113     }
00114
00115     /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */
00116     sys->pllcr = k_pllcr_hoco_x12;
00117     sys->pllcr2 = k_pllcr2_enable;
00118
00119     /* Step 4: wait for PLL to stabilise. */
00120     while ((sys->oscovfsr & k_oscovfsr_plovf) == 0U) {
00121         __asm__ volatile("nop");
00122     }
00123
00124     /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */

```



```

00125  *memwait_reg() = k_memwait_one;
00126
00127  /* Step 6: program all bus dividers. */
00128  sys->sckcr = k_sckcr_value;
00129
00130  /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131  *packcr_reg() = k_packcr_pll_source;
00132
00133  /* Step 8: program the USB clock divider. */
00134  sys->sckcr2 = k_sckcr2_uck_div4;
00135
00136  /* Step 9: switch the system clock source to PLL. */
00137  sys->sckcr3 = k_sckcr3_cksel_pll;
00138
00139  /* Step 10: relock PRCR. */
00140  *prcr_reg() = k_rx_prcr_lock;
00141 }

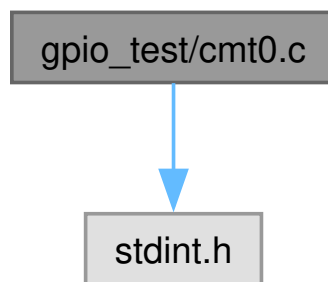
```

4.3 gpio_test/cmt0.c File Reference

CMT0 100 Hz tick source for ThreadX with PCKB = 48 MHz.

```
#include <stdint.h>
```

Include dependency graph for cmt0.c:



Enumerations

- enum `cmt_addr_t` : `uintptr_t` {
`k_cmstr0_addr` = 0x00088000U , `k_cmt0_cmcr` = 0x00088002U , `k_cmt0_cmcnt` = 0x00088004U
, `k_cmt0_cmcor` = 0x00088006U ,
`k_icu_ir_base` = 0x00087000U , `k_icu_ier_base` = 0x00087200U , `k_icu_ipr_base` =
0x00087300U , `k_mstpcra_addr` = 0x00080010U ,
`k_prcr_addr` = 0x000803FEU }

Absolute hardware register addresses used by the CMT0 tick setup.

- enum `cmt0_cfg_t` : `uint16_t` {
`k_cmt0_cmcor_val` = 14999U , `k_cmt0_cmcr_val` = 0x0041U , `k_cmstr0_ch0_bit` = 0x0001U ,
`k_prcr_unlock_prc0_prc1` = 0xA503U ,
`k_prcr_lock` = 0xA500U }

16-bit configuration values written to CMT0 and PRCR.

- enum `icu_cmt0_cfg_t` : `uint8_t` {
`k_vect_cmt0` = 28U , `k_cmt0_priority` = 5U , `k_icu_ier_cmt0` = 3U , `k_icu_ier_cmt0_bit` =
4U ,
`k_mstpa_cmt0_bit` = 15U }

ICU vector/priority/bit constants for CMT0 CMI0.

Functions

- void [__tx_timer_interrupt](#) (void)
- static volatile uint16_t * [internal_cmstr0](#) (void)
Typed accessor for CMSTR0 (CMT0/1 start register).
- static volatile uint16_t * [internal_cmt0_cmcr](#) (void)
Typed accessor for CMT0.CMCR.
- static volatile uint16_t * [internal_cmt0_cmcnt](#) (void)
Typed accessor for CMT0.CMCNT.
- static volatile uint16_t * [internal_cmt0_cmcpr](#) (void)
Typed accessor for CMT0.CMCOR.
- static volatile uint8_t * [internal_icu_ir](#) (uint8_t vect)
Typed accessor for ICU.IR[vect] (one-byte interrupt request flag).
- static volatile uint8_t * [internal_icu_ier](#) (uint8_t idx)
Typed accessor for ICU.IER[idx] (vect/8 selects the byte).
- static volatile uint8_t * [internal_icu_ipr](#) (uint8_t vect)
Typed accessor for ICU.IPR[vect] (one-byte priority).
- static volatile uint32_t * [internal_mstpcra](#) (void)
Typed accessor for MSTPCRA (module stop control A).
- static volatile uint16_t * [internal_prctr](#) (void)
Typed accessor for PRCTR (clock/module-stop write protect).
- void [cmt0_isr](#) (void)
CMT0 CMI0 ISR: clears IR and drives the ThreadX tick.
- void [cmt0_init](#) (void)
Program CMT0 for a 100 Hz tick and enable its interrupt.

4.3.1 Detailed Description

CMT0 100 Hz tick source for ThreadX with PCKB = 48 MHz.

After [clock_init\(\)](#) PCKB = 48 MHz. PCKB/32 = 1.5 MHz, divided by 15000 gives 100 Hz exactly. CMCOR = 14999, CMCR.CKS = 01 (PCKB/32), CMIE = 1.

Vector 28 (CMT0 CMI0) is wired in vectors.S to [cmt0_isr](#) below, which is marked `attribute((interrupt))` so GCC generates the RTE epilogue.

Register and bit references follow the RX72N Hardware Manual (R01UH0824EJ0111), chapters 13 (ICU), 15 (CMT), and 11 (module stop).

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [cmt0.c](#).

4.3.2 Enumeration Type Documentation

4.3.2.1 cmt0_cfg_t

enum [cmt0_cfg_t](#) : uint16_t

16-bit configuration values written to CMT0 and PRCR.

Enumerator

k_cmt0_cmcpr_val	(48 MHz / 32) / 100 Hz - 1 = 14999 for 100 Hz tick
k_cmt0_cmcr_val	CMCR: CMIE=1 (interrupt), CKS=01 (PCKB/32)
k_cmstr0_ch0_bit	CMSTR0 bit 0: CMT0 start/stop
k_prclr_unlock_prc0_prc1	Write PRCR=0xA503 to enable PRC0+PRC1 writes
k_prclr_lock	Write PRCR=0xA500 to relock PRC0+PRC1

Definition at line 45 of file [cmt0.c](#).

```
00045      : uint16_t {
00046  k_cmt0_cmcpr_val    = 14999U, /**< (48 MHz / 32) / 100 Hz - 1 = 14999 for 100 Hz tick */
00047  k_cmt0_cmcr_val     = 0x0041U, /**< CMCR: CMIE=1 (interrupt), CKS=01 (PCKB/32) */
00048  k_cmstr0_ch0_bit    = 0x0001U, /**< CMSTR0 bit 0: CMT0 start/stop */
00049  k_prclr_unlock_prc0_prc1 = 0xA503U, /**< Write PRCR=0xA503 to enable PRC0+PRC1 writes */
00050  k_prclr_lock        = 0xA500U, /**< Write PRCR=0xA500 to relock PRC0+PRC1 */
00051 } cmt0_cfg_t;
```

4.3.2.2 cmt_addr_t

enum [cmt_addr_t](#) : uintptr_t

Absolute hardware register addresses used by the CMT0 tick setup.

Enumerator

k_cmstr0_addr	CMSTR0: shared CMT0/CMT1 start register (Ch 15)
k_cmt0_cmcr	CMT0 CMCR: control register (Ch 15)
k_cmt0_cmcnt	CMT0 CMCNT: counter value (Ch 15)
k_cmt0_cmcpr	CMT0 CMCOR: compare-match target (Ch 15)
k_icu_ir_base	ICU.IR[] base: IR[vect] at +vect (Ch 13)
k_icu_ier_base	ICU.IER[] base: IER[vect/8] bit (vect%8) (Ch 13)
k_icu_ipr_base	ICU.IPR[] base: IPR[vect] priority (Ch 13)
k_mstpcra_addr	MSTPCRA: module stop control A (Ch 11)
k_prclr_addr	PRCLR: protect register for clock/module stop writes

Definition at line 27 of file [cmt0.c](#).

```
00027      : uintptr_t {
00028  k_cmstr0_addr = 0x00088000U, /**< CMSTR0: shared CMT0/CMT1 start register (Ch 15) */
00029  k_cmt0_cmcr   = 0x00088002U, /**< CMT0 CMCR: control register (Ch 15) */
00030  k_cmt0_cmcnt  = 0x00088004U, /**< CMT0 CMCNT: counter value (Ch 15) */
00031  k_cmt0_cmcpr  = 0x00088006U, /**< CMT0 CMCOR: compare-match target (Ch 15) */
00032
00033  k_icu_ir_base = 0x00087000U, /**< ICU.IR[] base: IR[vect] at +vect (Ch 13) */
00034  k_icu_ier_base = 0x00087200U, /**< ICU.IER[] base: IER[vect/8] bit (vect%8) (Ch 13) */
00035  k_icu_ipr_base = 0x00087300U, /**< ICU.IPR[] base: IPR[vect] priority (Ch 13) */
00036
00037  k_mstpcra_addr = 0x00080010U, /**< MSTPCRA: module stop control A (Ch 11) */
00038  k_prclr_addr   = 0x000803FEU, /**< PRCR: protect register for clock/module stop writes */
00039 } cmt_addr_t;
```

4.3.2.3 icu_cmt0_cfg_t

enum `icu_cmt0_cfg_t` : `uint8_t`

ICU vector/priority/bit constants for CMT0 CMI0.

Enumerator

<code>k_vect_cmt0</code>	ICU vector number for CMT0 CMI0 (Ch 13)
<code>k_cmt0_priority</code>	IPR priority (0..15); 5 is low-enough for ThreadX tick
<code>k_icu_ier_cmt0</code>	IER register index for vector 28 ($28/8 = 3$)
<code>k_icu_ier_cmt0_bit</code>	Bit position within IER[3] ($28 \% 8 = 4$)
<code>k_mstpa_cmt0_bit</code>	MSTPCRA bit 15: combined CMT0/CMT1 module stop

Definition at line 57 of file `cmt0.c`.

```
00057 : uint8_t {
00058   k_vect_cmt0    = 28U, /**< ICU vector number for CMT0 CMI0 (Ch 13) */
00059   k_cmt0_priority = 5U,  /**< IPR priority (0..15); 5 is low-enough for ThreadX tick */
00060   k_icu_ier_cmt0  = 3U,  /**< IER register index for vector 28 (28/8 = 3) */
00061   k_icu_ier_cmt0_bit = 4U, /**< Bit position within IER[3] (28 % 8 = 4) */
00062   k_mstpa_cmt0_bit = 15U, /**< MSTPCRA bit 15: combined CMT0/CMT1 module stop */
00063 } icu_cmt0_cfg_t;
```

4.3.3 Function Documentation

4.3.3.1 __tx_timer_interrupt()

```
void __tx_timer_interrupt (
    void ) [extern]
```

Referenced by `cmt0_isr()`.

Here is the caller graph for this function:



4.3.3.2 cmt0_init()

```
void cmt0_init (
    void )
```

Program CMT0 for a 100 Hz tick and enable its interrupt.

1. Releases CMT0 (and CMT1) from module-stop via MSTPCRA bit 15, bracketed by the PRCR unlock/relock protocol.
2. Stops CMT0, programs CMCR/CMCOR, clears CMCNT.
3. Clears any pending IR, sets IPR=5, enables the vector in IER.
4. Starts CMT0 and globally enables interrupts with SETPSW I.

Precondition

[clock_init\(\)](#) has completed so PCKB = 48 MHz.
 Called from [main\(\)](#) before [tx_kernel_enter\(\)](#).

Postcondition

CMT0 fires at exactly 100 Hz into [cmt0_isr](#).
 Global interrupt flag (PSW.I) is set.

Definition at line 165 of file [cmt0.c](#).

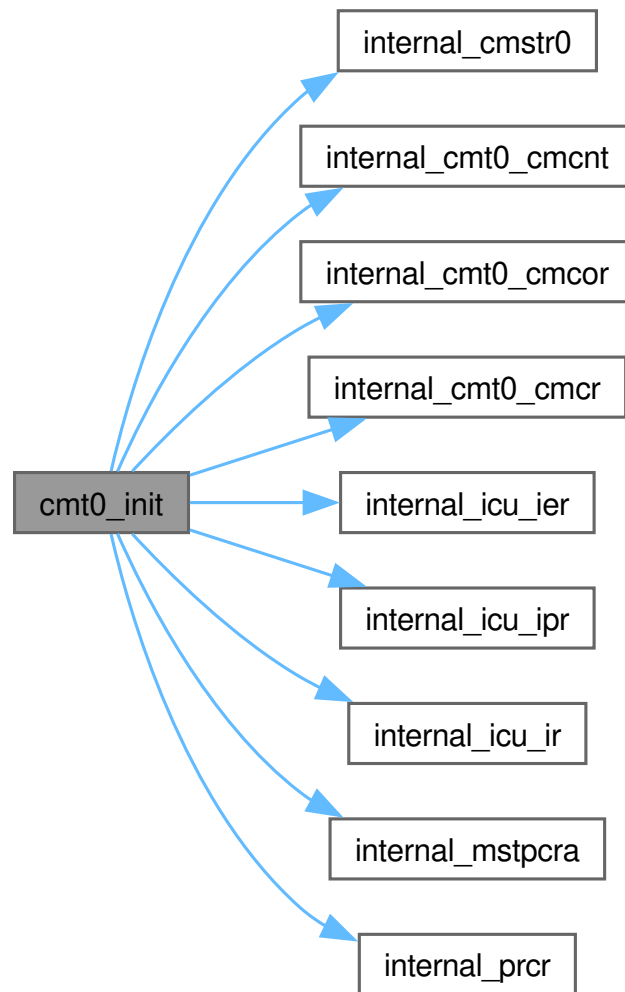
```

00166 {
00167     /* Release CMT0/CMT1 from module stop (MSTPCRA bit 15 = 0) under PRCR protection. */
00168     *internal_prcr() = k_prcr_unlock_prc0_prc1;
00169     *internal_mstpcra() &= ~(uint32_t)(1UL « k_mstpa_cmt0_bit);
00170     *internal_prcr() = k_prcr_lock;
00171
00172     /* Stop CMT0 before reprogramming. */
00173     *internal_cmstr0() &= (uint16_t) ~(uint16_t) k_cmstr0_ch0_bit;
00174
00175     *internal_cmt0_cmcr() = k_cmt0_cmcr_val;
00176     *internal_cmt0_cmcor() = k_cmt0_cmcor_val;
00177     *internal_cmt0_cmcnt() = 0U;
00178
00179     *internal_icu_ir(k_vect_cmt0) = 0U;
00180     *internal_icu_ipr(k_vect_cmt0) = k_cmt0_priority;
00181     *internal_icu_ier(k_icu_ier_cmt0) |= (uint8_t)(1U « k_icu_ier_cmt0_bit);
00182
00183     *internal_cmstr0() |= k_cmstr0_ch0_bit;
00184
00185     /* Inline asm: RX has no standalone GCC builtin for SETPSW; this is the
00186      * idiomatic way to set PSW.I (global interrupt enable) at runtime. */
00187     __asm__ volatile("setpsw i");
00188 }
```

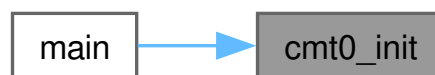
References [internal_cmstr0\(\)](#), [internal_cmt0_cmcnt\(\)](#), [internal_cmt0_cmcor\(\)](#), [internal_cmt0_cmcr\(\)](#), [internal_icu_ier\(\)](#), [internal_icu_ipr\(\)](#), [internal_icu_ir\(\)](#), [internal_mstpcra\(\)](#), [internal_prcr\(\)](#), [k_cmstr0_ch0_bit](#), [k_cmt0_cmcor_val](#), [k_cmt0_cmcr_val](#), [k_cmt0_priority](#), [k_icu_ier_cmt0](#), [k_icu_ier_cmt0_bit](#), [k_mstpa_cmt0_bit](#), [k_prcr_lock](#), [k_prcr_unlock_prc0_prc1](#), and [k_vect_cmt0](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.3 cmt0_isr()

```
void cmt0_isr (
    void )
```

CMT0 CMI0 ISR: clears IR and drives the ThreadX tick.

Wired by vectors.S into slot 28 of the relocatable vector table. Marked `__attribute__((interrupt))` so GCC-RX emits the RTE prologue and epilogue around the body.

Precondition

Interrupts are globally enabled and this vector is wired in INTB.

Postcondition

[_tx_timer_interrupt\(\)](#) has advanced the ThreadX system clock by exactly one tick.

Note

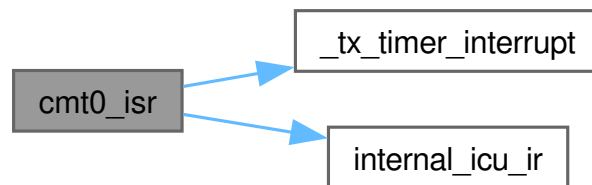
Executes in ISR context on the ISP.

Definition at line 144 of file [cmt0.c](#).

```
00145 {
00146     *internal_icu_ir(k_vect_cmt0) = 0U;
00147     _tx_timer_interrupt();
00148 }
```

References [_tx_timer_interrupt\(\)](#), [internal_icu_ir\(\)](#), and [k_vect_cmt0](#).

Here is the call graph for this function:



4.3.3.4 internal_cmstr0()

```
volatile uint16_t * internal_cmstr0 (
    void ) [inline], [static]
```

Typed accessor for CMSTR0 (CMT0/1 start register).

Definition at line 66 of file [cmt0.c](#).

```
00067 {
00068     return (volatile uint16_t *)k_cmstr0_addr;
00069 }
```

References [k_cmstr0_addr](#).

Referenced by [cmt0_init\(\)](#).

Here is the caller graph for this function:



4.3.3.5 `internal_cmt0_cmcnt()`

```
volatile uint16_t * internal_cmt0_cmcnt (
    void ) [inline], [static]
```

Typed accessor for CMT0.CMCNT.

Definition at line 78 of file `cmt0.c`.

```
00079 {
00080     return (volatile uint16_t *)k_cmt0_cmcnt;
00081 }
```

References [k_cmt0_cmcnt](#).

Referenced by [cmt0_init\(\)](#).

Here is the caller graph for this function:



4.3.3.6 `internal_cmt0_cmcor()`

```
volatile uint16_t * internal_cmt0_cmcor (
    void ) [inline], [static]
```

Typed accessor for CMT0.CMCOR.

Definition at line 84 of file `cmt0.c`.

```
00085 {
00086     return (volatile uint16_t *)k_cmt0_cmcor;
00087 }
```

References [k_cmt0_cmcor](#).

Referenced by [cmt0_init\(\)](#).

Here is the caller graph for this function:



4.3.3.7 `internal_cmt0_cmcr()`

```
volatile uint16_t * internal_cmt0_cmcr (
    void ) [inline], [static]
```

Typed accessor for CMT0.CMCR.

Definition at line 72 of file `cmt0.c`.

```
00073 {
00074     return (volatile uint16_t *)k_cmt0_cmcr;
00075 }
```

References [k_cmt0_cmcr](#).

Referenced by [cmt0_init\(\)](#).

Here is the caller graph for this function:



4.3.3.8 internal_icu_ier()

```
volatile uint8_t * internal_icu_ier (
    uint8_t idx)    [inline], [static]
```

Typed accessor for ICU.IER[idx] (vect/8 selects the byte).

Parameters

in	idx	IER register index (vect / 8).
----	-----	--------------------------------

Returns

Volatile pointer to IER[idx].

Definition at line 104 of file [cmt0.c](#).

```
00105 {
00106     return (volatile uint8_t *) (k_icu_ier_base + idx);
00107 }
```

References [k_icu_ier_base](#).

Referenced by [cmt0_init\(\)](#).

Here is the caller graph for this function:



4.3.3.9 internal_icu_ipr()

```
volatile uint8_t * internal_icu_ipr (
    uint8_t vect)    [inline], [static]
```

Typed accessor for ICU.IPR[vect] (one-byte priority).

Parameters

in	vect	ICU vector number (0..255).
----	------	-----------------------------

Returns

Volatile pointer to IPR[vect].

Definition at line 114 of file [cmt0.c](#).

```
00115 {
00116     return (volatile uint8_t *) (k_icu_ipr_base + vect);
00117 }
```

References [k_icu_ipr_base](#).

Referenced by [cmt0_init\(\)](#).

Here is the caller graph for this function:



4.3.3.10 internal_icu_ir()

```
volatile uint8_t * internal_icu_ir (
    uint8_t vect)    [inline], [static]
```

Typed accessor for ICU.IR[vect] (one-byte interrupt request flag).

Parameters

in	vect	ICU vector number (0..255).
----	------	-----------------------------

Returns

Volatile pointer to IR[vect].

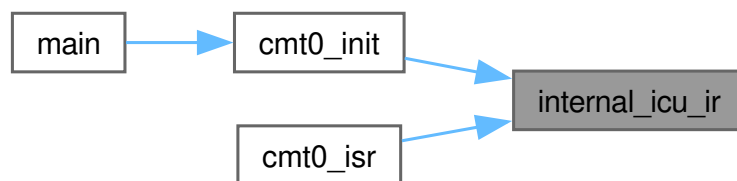
Definition at line 94 of file [cmt0.c](#).

```
00095 {
00096     return (volatile uint8_t *) (k_icu_ir_base + vect);
00097 }
```

References [k_icu_ir_base](#).

Referenced by [cmt0_init\(\)](#), and [cmt0_isr\(\)](#).

Here is the caller graph for this function:



4.3.3.11 internal_mstpcra()

```
volatile uint32_t * internal_mstpcra (
    void )    [inline], [static]
```

Typed accessor for MSTPCRA (module stop control A).

Definition at line 120 of file [cmt0.c](#).

```
00121 {
00122     return (volatile uint32_t *) k_mstpcra_addr;
00123 }
```

References [k_mstpcra_addr](#).

Referenced by [cmt0_init\(\)](#).

Here is the caller graph for this function:



4.3.3.12 internal_prcr()

```
volatile uint16_t * internal_prcr (
    void ) [inline], [static]
```

Typed accessor for PRCR (clock/module-stop write protect).

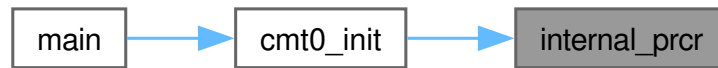
Definition at line 126 of file [cmt0.c](#).

```
00127 {
00128     return (volatile uint16_t *)k_prcr_addr;
00129 }
```

References [k_prcr_addr](#).

Referenced by [cmt0_init\(\)](#).

Here is the caller graph for this function:



4.4 cmt0.c

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file cmt0.c
00003  * @brief CMT0 100 Hz tick source for ThreadX with PCKB = 48 MHz.
00004  *
00005  * @details
00006  * After clock_init() PCKB = 48 MHz. PCKB/32 = 1.5 MHz, divided by 15000
00007  * gives 100 Hz exactly. CMCOR = 14999, CMCR.CKS = 01 (PCKB/32), CMIE = 1.
00008  *
00009  * Vector 28 (CMT0 CMI0) is wired in vectors.S to cmt0_isr below, which is
00010  * marked __attribute__((interrupt)) so GCC generates the RTE epilogue.
00011  *
00012  * Register and bit references follow the RX72N Hardware Manual
00013  * (R01UH0824EJ0111), chapters 13 (ICU), 15 (CMT), and 11 (module stop).
00014  *
00015  * SPDX-License-Identifier: MIT
00016  * @copyright Copyright (c) 2026 Locked Inc.
00017  */
00018
00019 #include <stdint.h>
00020
00021 extern void __tx_timer_interrupt(void);
00022
00023 /**
00024  * @enum cmt_addr_t
00025  * @brief Absolute hardware register addresses used by the CMT0 tick setup.
00026  */
00027 typedef enum : uintptr_t {
00028     k_cmstr0_addr = 0x00088000U, /**< CMSTR0: shared CMT0/CMT1 start register (Ch 15) */
00029     k_cmt0_cmcr   = 0x00088002U, /**< CMT0 CMCR: control register (Ch 15) */
00030     k_cmt0_cmcnt  = 0x00088004U, /**< CMT0 CMCNT: counter value (Ch 15) */
00031     k_cmt0_cmcpr  = 0x00088006U, /**< CMT0 CMCOR: compare-match target (Ch 15) */
00032
00033     k_icu_ir_base = 0x00087000U, /**< ICU.IR[] base: IR[vect] at +vect (Ch 13) */
00034     k_icu_ier_base = 0x00087200U, /**< ICU.IER[] base: IER[vect/8] bit (vect%8) (Ch 13) */
00035     k_icu_ipr_base = 0x00087300U, /**< ICU.IPR[] base: IPR[vect] priority (Ch 13) */
00036
00037     k_mstpcra_addr = 0x00080010U, /**< MSTPCRA: module stop control A (Ch 11) */
00038     k_prcr_addr    = 0x000803FEU, /**< PRCR: protect register for clock/module stop writes */
00039 } cmt_addr_t;
00040
00041 /**
00042  * @enum cmt0_cfg_t
00043  * @brief 16-bit configuration values written to CMT0 and PRCR.
00044  */
00045 typedef enum : uint16_t {
00046     k_cmt0_cmcpr_val = 14999U, /**< (48 MHz / 32) / 100 Hz - 1 = 14999 for 100 Hz tick */
```

```

00047     k_cmt0_cmcr_val      = 0x0041U, /**< CMCR: CMIE=1 (interrupt), CKS=01 (PCKB/32) */
00048     k_cmstr0_ch0_bit     = 0x0001U, /**< CMSTR0 bit 0: CMT0 start/stop */
00049     k_prcr_unlock_prc0_prc1 = 0xA503U, /**< Write PRCR=0xA503 to enable PRC0+PRC1 writes */
00050     k_prcr_lock          = 0xA500U, /**< Write PRCR=0xA500 to relock PRC0+PRC1 */
00051 } cmt0_cfg_t;
00052
00053 /**
00054  * @enum icu_cmt0_cfg_t
00055  * @brief ICU vector/priority/bit constants for CMT0 CMI0.
00056  */
00057 typedef enum : uint8_t {
00058     k_vect_cmt0      = 28U, /**< ICU vector number for CMT0 CMI0 (Ch 13) */
00059     k_cmt0_priority  = 5U,  /**< IPR priority (0..15); 5 is low-enough for ThreadX tick */
00060     k_icu_ier_cmt0   = 3U,  /**< IER register index for vector 28 (28/8 = 3) */
00061     k_icu_ier_cmt0_bit = 4U, /**< Bit position within IER[3] (28 % 8 = 4) */
00062     k_mstpa_cmt0_bit  = 15U, /**< MSTPCRA bit 15: combined CMT0/CMT1 module stop */
00063 } icu_cmt0_cfg_t;
00064
00065 /** @brief Typed accessor for CMSTR0 (CMT0/1 start register). */
00066 static inline volatile uint16_t *internal_cmstr0(void)
00067 {
00068     return (volatile uint16_t *)k_cmstr0_addr;
00069 }
00070
00071 /** @brief Typed accessor for CMT0.CMCR. */
00072 static inline volatile uint16_t *internal_cmt0_cmcr(void)
00073 {
00074     return (volatile uint16_t *)k_cmt0_cmcr;
00075 }
00076
00077 /** @brief Typed accessor for CMT0.CMCNT. */
00078 static inline volatile uint16_t *internal_cmt0_cmcnt(void)
00079 {
00080     return (volatile uint16_t *)k_cmt0_cmcnt;
00081 }
00082
00083 /** @brief Typed accessor for CMT0.CMCOR. */
00084 static inline volatile uint16_t *internal_cmt0_cmcpr(void)
00085 {
00086     return (volatile uint16_t *)k_cmt0_cmcpr;
00087 }
00088
00089 /**
00090  * @brief Typed accessor for ICU.IR[vect] (one-byte interrupt request flag).
00091  * @param[in] vect ICU vector number (0..255).
00092  * @return Volatile pointer to IR[vect].
00093  */
00094 static inline volatile uint8_t *internal_icu_ir(uint8_t vect)
00095 {
00096     return (volatile uint8_t *) (k_icu_ir_base + vect);
00097 }
00098
00099 /**
00100  * @brief Typed accessor for ICU.IER[idx] (vect/8 selects the byte).
00101  * @param[in] idx IER register index (vect / 8).
00102  * @return Volatile pointer to IER[idx].
00103  */
00104 static inline volatile uint8_t *internal_icu_ier(uint8_t idx)
00105 {
00106     return (volatile uint8_t *) (k_icu_ier_base + idx);
00107 }
00108
00109 /**
00110  * @brief Typed accessor for ICU.IPR[vect] (one-byte priority).
00111  * @param[in] vect ICU vector number (0..255).
00112  * @return Volatile pointer to IPR[vect].
00113  */
00114 static inline volatile uint8_t *internal_icu_ipr(uint8_t vect)
00115 {
00116     return (volatile uint8_t *) (k_icu_ipr_base + vect);
00117 }
00118
00119 /** @brief Typed accessor for MSTPCRA (module stop control A). */
00120 static inline volatile uint32_t *internal_mstpcra(void)
00121 {
00122     return (volatile uint32_t *)k_mstpcra_addr;
00123 }
00124
00125 /** @brief Typed accessor for PRCR (clock/module-stop write protect). */
00126 static inline volatile uint16_t *internal_prcr(void)
00127 {
00128     return (volatile uint16_t *)k_prcr_addr;
00129 }
00130
00131 /**
00132  * @brief CMT0 CMI0 ISR: clears IR and drives the ThreadX tick.
00133  */

```

```

00134 * @details
00135 * Wired by `vectors.S` into slot 28 of the relocatable vector table.
00136 * Marked `__attribute__((interrupt))` so GCC-RX emits the RTE prologue
00137 * and epilogue around the body.
00138 *
00139 * @pre Interrupts are globally enabled and this vector is wired in INTB.
00140 * @post `_tx_timer_interrupt()` has advanced the ThreadX system clock by
00141 * exactly one tick.
00142 * @note Executes in ISR context on the ISP.
00143 */
00144 void __attribute__((interrupt)) cmt0_isr(void)
00145 {
00146     *internal_icu_ir(k_vect_cmt0) = 0U;
00147     _tx_timer_interrupt();
00148 }
00149
00150 /**
00151 * @brief Program CMT0 for a 100 Hz tick and enable its interrupt.
00152 *
00153 * @details
00154 * 1. Releases CMT0 (and CMT1) from module-stop via MSTPCRA bit 15,
00155 * bracketed by the PRCR unlock/relock protocol.
00156 * 2. Stops CMT0, programs CMCR/CMCOR, clears CMCNT.
00157 * 3. Clears any pending IR, sets IPR=5, enables the vector in IER.
00158 * 4. Starts CMT0 and globally enables interrupts with `SETPSW I`.
00159 *
00160 * @pre `clock_init()` has completed so PCKB = 48 MHz.
00161 * @pre Called from `main()` before `tx_kernel_enter()`.
00162 * @post CMT0 fires at exactly 100 Hz into `cmt0_isr`.
00163 * @post Global interrupt flag (PSW.I) is set.
00164 */
00165 void cmt0_init(void)
00166 {
00167     /* Release CMT0/CMT1 from module stop (MSTPCRA bit 15 = 0) under PRCR protection. */
00168     *internal_prcr() = k_prcr_unlock_prc0_prc1;
00169     *internal_mstpcra() &= ~(uint32_t)(1UL « k_mstpa_cmt0_bit);
00170     *internal_prcr() = k_prcr_lock;
00171
00172     /* Stop CMT0 before reprogramming. */
00173     *internal_cmstr0() &= (uint16_t) ~(uint16_t) k_cmstr0_ch0_bit;
00174
00175     *internal_cmt0_cmcr() = k_cmt0_cmcr_val;
00176     *internal_cmt0_cmcpr() = k_cmt0_cmcpr_val;
00177     *internal_cmt0_cmcnt() = 0U;
00178
00179     *internal_icu_ir(k_vect_cmt0) = 0U;
00180     *internal_icu_ipr(k_vect_cmt0) = k_cmt0_priority;
00181     *internal_icu_ier(k_icu_ier_cmt0) |= (uint8_t)(1U « k_icu_ier_cmt0_bit);
00182
00183     *internal_cmstr0() |= k_cmstr0_ch0_bit;
00184
00185     /* Inline asm: RX has no standalone GCC builtin for SETPSW; this is the
00186      * idiomatic way to set PSW.I (global interrupt enable) at runtime. */
00187     __asm__ volatile("setpsw i");
00188 }

```

4.5 gpio_test/linker.ld File Reference

4.6 linker.ld

[Go to the documentation of this file.](#)

```

00001 /*
00002 * gpio_test/linker.ld -- Linker script for GPIO breakout board test.
00003 *
00004 * Same memory map as usb_test: 512 KB internal SRAM, 2 MB internal ROM.
00005 * Includes Renesas-syntax sections (P, B, D, C) needed by ThreadX RXv3 port.
00006 */
00007
00008 MEMORY
00009 {
00010     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC /* 512 KB - 4 bytes */
00011     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000 /* 2 MB */
00012 }
00013
00014 SECTIONS
00015 {
00016     /* Code -- explicit ROM base so section VMA is anchored correctly. */
00017     .text 0xFFE00000 : AT(0xFFE00000)

```

```

00018 {
00019     *(.text*)
00020     *(P)
00021     *(P_1)
00022     *(.rodata*)
00023     *(C)
00024     *(C_1)
00025     *(C_2)
00026     etext = .;
00027 } > ROM
00028
00029 /*
00030  * Relocatable vector table -- startup.S loads INTB with this address.
00031  * Must be 4-byte aligned; only vectors 27 (SWINT) and 28 (CMT0) are
00032  * used, but allocate 256 entries (1024 bytes) for safety.
00033  */
00034 .rvecvectors ALIGN(4) :
00035 {
00036     _rvecvectors_start = .;
00037     KEEP(*(.rvecvectors))
00038     _rvecvectors_end = .;
00039 } > ROM
00040
00041 /*
00042  * Initialised data -- LMA stays in ROM, VMA in RAM.
00043  * startup.S copies _data..edata from _mdata on boot.
00044  */
00045 _mdata = .;
00046 .data : AT(_mdata)
00047 {
00048     _data = .;
00049     *(.data*)
00050     *(D)
00051     *(D_1)
00052     *(D_2)
00053     _edata = .;
00054 } > RAM
00055
00056 /* BSS -- zeroed by startup.S */
00057 .bss :
00058 {
00059     _bss = .;
00060     *(.bss*)
00061     *(COMMON)
00062     *(B)
00063     *(B_1)
00064     *(B_2)
00065     _ebss = .;
00066     . = ALIGN(4);
00067     _end = .;
00068 } > RAM AT>RAM
00069
00070 /*
00071  * Stacks -- USP grows down from top of RAM, ISP 4 KB below that.
00072  */
00073 _ustack = ORIGIN(RAM) + LENGTH(RAM);
00074 _istack = _ustack - 0x1000;
00075
00076 /*
00077  * Exception vector table at fixed hardware address 0xFFFFF80.
00078  * 32 entries -- all unused.
00079  */
00080 .exvectors 0xFFFFF80 : AT(0xFFFFF80)
00081 {
00082     LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF);
00083     LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF);
00084     LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF);
00085     LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF);
00086     LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF);
00087     LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF);
00088     LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF);
00089     LONG(0xFFFFFFF); LONG(0xFFFFFFF); LONG(0xFFFFFFF);
00090 } > ROM
00091
00092 /* Fixed reset vector at 0xFFFFF8C */
00093 .fvectors 0xFFFFF8C : AT(0xFFFFF8C)
00094 {
00095     KEEP(*(.fvectors))
00096 } > ROM
00097 }

```

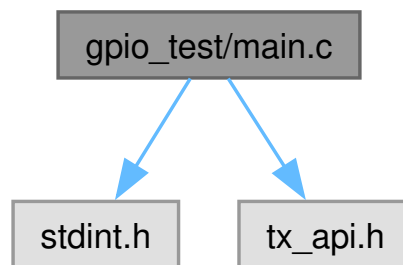
4.7 gpio_test/main.c File Reference

GPIO breakout verification firmware for RX72N, ThreadX edition.

```
#include <stdint.h>
```

```
#include "tx_api.h"
```

Include dependency graph for main.c:



Data Structures

- struct `pin_desc_t`
One entry in the firmware's pin sweep table.

Enumerations

- enum `port_reg_base_t` : `uintptr_t` { `k_pdr_base` = 0x0008C000U , `k_podr_base` = 0x0008C020U , `k_pmr_base` = 0x0008C060U }
Base addresses of the per-port GPIO register banks.
- enum `port_offset_t` : `uint8_t` {
`k_port_off_0` = 0x00U , `k_port_off_1` = 0x01U , `k_port_off_2` = 0x02U , `k_port_off_3` = 0x03U ,
`k_port_off_4` = 0x04U , `k_port_off_5` = 0x05U , `k_port_off_6` = 0x06U , `k_port_off_7` = 0x07U ,
`k_port_off_8` = 0x08U , `k_port_off_9` = 0x09U , `k_port_off_a` = 0x0AU , `k_port_off_b` = 0x0BU ,
`k_port_off_c` = 0x0CU , `k_port_off_d` = 0x0DU , `k_port_off_e` = 0x0EU , `k_port_off_f` = 0x0FU }
Per-port index added to every GPIO register bank base.
- enum `port_bit_t` : `uint8_t` {
`k_bit_0` = 0U , `k_bit_1` = 1U , `k_bit_2` = 2U , `k_bit_3` = 3U ,
`k_bit_4` = 4U , `k_bit_5` = 5U , `k_bit_6` = 6U , `k_bit_7` = 7U }
Bit position within an 8-bit port register.
- enum `timing_t` : `uint32_t` { `k_num_pins` = (uint32_t)(sizeof(s_pins) / sizeof(s_pins[0])) , `k_delay_iter` = 50000U , `k_gap_iters` = 500U }
Sweep loop iteration and gap counts.
- enum `rtos_prio_t` : `uint8_t` { `k_sweep_priority` = 10U }
ThreadX priority numbers used by this firmware.
- enum `rtos_stack_t` : `uint16_t` { `k_sweep_stack_sz` = 512U }
ThreadX stack sizes in bytes.

Functions

- static volatile uint8_t * [internal_pdr](#) (uint8_t port_off)
Pointer to the PDR (direction) register for a given port offset.
- static volatile uint8_t * [internal_podr](#) (uint8_t port_off)
Pointer to the PODR (output data) register for a given port offset.
- static volatile uint8_t * [internal_pmr](#) (uint8_t port_off)
Pointer to the PMR (mode) register for a given port offset.
- static void [internal_delay](#) (void)
Busy-wait for one pin state (~5 ms at ICLK 96 MHz).
- static void [internal_gpio_init_all](#) (void)
Configure every pin in [s_pins](#) as a GPIO output driven LOW.
- static void [internal_sweep_task_entry](#) (ULONG arg)
Sweep task: pulses each pin in [s_pins](#) order, forever.
- void [tx_application_define](#) (void *first_unused_memory)
ThreadX hook: configure GPIO and start the sweep task.
- void [clock_init](#) (void)
- void [cmt0_init](#) (void)
Program CMT0 for a 100 Hz tick and enable its interrupt.
- int [main](#) (void)
Firmware entry point.

Variables

- static const [pin_desc_t](#) [s_pins](#) []
Full sweep table, one entry per tested GPIO pin.
- static TX_THREAD [s_sweep_tcb](#)
- static uint8_t [s_sweep_stack](#) [[k_sweep_stack_sz](#)]

4.7.1 Detailed Description

GPIO breakout verification firmware for RX72N, ThreadX edition.

Mirrors the structure that works in `blinky_rtos`: one ThreadX thread that drives the pins with a busy-wait delay (NOT `tx_thread_sleep`, which stalls when the scheduler/tick isn't ticking for any reason). A single 100 Hz CMT0 tick keeps the kernel happy even though we do not use it for sleep-based timing.

Test pattern per pin:

1. Drive HIGH for ~5 ms (busy-wait loop)
2. Drive LOW for ~5 ms
3. Advance to next pin

After all pins are tested a long quiet gap (~2.3 s) separates cycles so the host script can anchor capture timing to the gap.

Clock path: [clock_init\(\)](#) brings us from HOCO 16 MHz -> PLL 192 MHz -> ICLK 96 MHz / PCKB 48 MHz. CMT0 is then programmed at PCKB/32 for 100 Hz tick.

PJ3 and PJ5 are excluded: on RX72N they double as JTAG TMS/TDO and driving them while the E2 Lite has the debug interface latched hangs the MCU.

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [main.c](#).

4.7.2 Data Structure Documentation

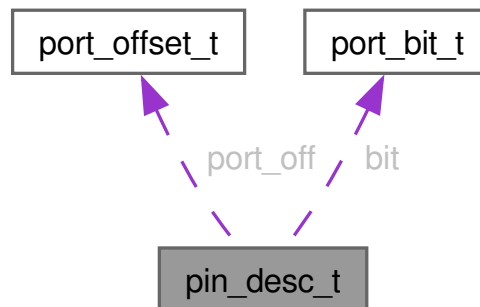
4.7.2.1 struct pin_desc_t

One entry in the firmware's pin sweep table.

The sweep iterates `s_pins[]` in order; its index is the firmware pin index matched by the host capture script.

Definition at line 121 of file `main.c`.

Collaboration diagram for `pin_desc_t`:



Data Fields

<code>port_bit_t</code>	bit	Bit position (0..7) within the 8-bit port register
<code>port_offset_t</code>	port_off	Port offset, used with <code>k_pdr_base</code> / <code>k_podr_base</code> / <code>k_pmr_base</code>

4.7.3 Enumeration Type Documentation

4.7.3.1 port_bit_t

```
enum port_bit_t : uint8_t
```

Bit position within an 8-bit port register.

RX72N ports are 8 bits wide; pin N of a given port is bit N of the port's PDR/PODR/PMR. Using a typed enum in `s_pins[]` keeps the table readable and satisfies the STAR coding rule against raw numeric literals.

Enumerator

<code>k_bit_0</code>	Bit 0 (pin 0) of the port
<code>k_bit_1</code>	Bit 1 (pin 1) of the port
<code>k_bit_2</code>	Bit 2 (pin 2) of the port
<code>k_bit_3</code>	Bit 3 (pin 3) of the port
<code>k_bit_4</code>	Bit 4 (pin 4) of the port
<code>k_bit_5</code>	Bit 5 (pin 5) of the port

k_bit_6	Bit 6 (pin 6) of the port
k_bit_7	Bit 7 (pin 7) of the port

Definition at line 99 of file [main.c](#).

```

00099      : uint8_t {
00100      k_bit_0 = 0U, /**< Bit 0 (pin 0) of the port */
00101      k_bit_1 = 1U, /**< Bit 1 (pin 1) of the port */
00102      k_bit_2 = 2U, /**< Bit 2 (pin 2) of the port */
00103      k_bit_3 = 3U, /**< Bit 3 (pin 3) of the port */
00104      k_bit_4 = 4U, /**< Bit 4 (pin 4) of the port */
00105      k_bit_5 = 5U, /**< Bit 5 (pin 5) of the port */
00106      k_bit_6 = 6U, /**< Bit 6 (pin 6) of the port */
00107      k_bit_7 = 7U, /**< Bit 7 (pin 7) of the port */
00108 } port_bit_t;

```

4.7.3.2 port_offset_t

```
enum port_offset_t : uint8_t
```

Per-port index added to every GPIO register bank base.

Ports 0..9 map to offsets 0x00..0x09, ports A..F to 0x0A..0x0F. Port J lives at offset 0x12 because offsets 0x10 (Port G) and 0x11 (Port H) are reserved on this package even though the address space is allocated.

Enumerator

k_port_off_0	Port 0 offset within each register bank
k_port_off_1	Port 1 offset within each register bank
k_port_off_2	Port 2 offset within each register bank
k_port_off_3	Port 3 offset within each register bank
k_port_off_4	Port 4 offset within each register bank
k_port_off_5	Port 5 offset within each register bank
k_port_off_6	Port 6 offset within each register bank
k_port_off_7	Port 7 offset within each register bank
k_port_off_8	Port 8 offset within each register bank
k_port_off_9	Port 9 offset within each register bank
k_port_off_a	Port A offset within each register bank
k_port_off_b	Port B offset within each register bank
k_port_off_c	Port C offset within each register bank
k_port_off_d	Port D offset within each register bank
k_port_off_e	Port E offset within each register bank
k_port_off_f	Port F offset within each register bank

Definition at line 68 of file [main.c](#).

```

00068      : uint8_t {
00069      k_port_off_0 = 0x00U, /**< Port 0 offset within each register bank */
00070      k_port_off_1 = 0x01U, /**< Port 1 offset within each register bank */
00071      k_port_off_2 = 0x02U, /**< Port 2 offset within each register bank */
00072      k_port_off_3 = 0x03U, /**< Port 3 offset within each register bank */
00073      k_port_off_4 = 0x04U, /**< Port 4 offset within each register bank */
00074      k_port_off_5 = 0x05U, /**< Port 5 offset within each register bank */
00075      k_port_off_6 = 0x06U, /**< Port 6 offset within each register bank */
00076      k_port_off_7 = 0x07U, /**< Port 7 offset within each register bank */
00077      k_port_off_8 = 0x08U, /**< Port 8 offset within each register bank */

```

```

00078     k_port_off_9 = 0x09U, /**< Port 9 offset within each register bank */
00079     k_port_off_a = 0x0AU, /**< Port A offset within each register bank */
00080     k_port_off_b = 0x0BU, /**< Port B offset within each register bank */
00081     k_port_off_c = 0x0CU, /**< Port C offset within each register bank */
00082     k_port_off_d = 0x0DU, /**< Port D offset within each register bank */
00083     k_port_off_e = 0x0EU, /**< Port E offset within each register bank */
00084     k_port_off_f = 0x0FU, /**< Port F offset within each register bank */
00085 } port_offset_t;

```

4.7.3.3 port_reg_base_t

```
enum port_reg_base_t : uintptr_t
```

Base addresses of the per-port GPIO register banks.

Each GPIO port has three 8-bit registers indexed by [port_offset_t](#):

- PDR (direction): 0 = input, 1 = output
- PODR (output data): 0 = LOW, 1 = HIGH
- PMR (mode): 0 = GPIO, 1 = peripheral Address = base + port_offset.

See also

RX72N Hardware Manual, Chapter 22 (I/O Ports)

Enumerator

k_pdr_base	Port direction register bank base
k_podr_base	Port output data register bank base
k_pmr_base	Port mode register bank base

Definition at line 50 of file [main.c](#).

```

00050         : uintptr_t {
00051     k_pdr_base = 0x0008C000U, /**< Port direction register bank base */
00052     k_podr_base = 0x0008C020U, /**< Port output data register bank base */
00053     k_pmr_base = 0x0008C060U, /**< Port mode register bank base */
00054 } port_reg_base_t;

```

4.7.3.4 rtos_prio_t

```
enum rtos_prio_t : uint8_t
```

ThreadX priority numbers used by this firmware.

Enumerator

k_sweep_priority	Priority of the gpio_sweep thread
------------------	-----------------------------------

Definition at line 198 of file [main.c](#).

```

00198         : uint8_t {
00199     k_sweep_priority = 10U, /**< Priority of the gpio_sweep thread */
00200 } rtos_prio_t;

```

4.7.3.5 rtos_stack_t

enum [rtos_stack_t](#) : uint16_t

ThreadX stack sizes in bytes.

Enumerator

k_sweep_stack_sz	gpio_sweep thread stack size
------------------	------------------------------

Definition at line 206 of file [main.c](#).

```
00206      : uint16_t {
00207    k_sweep_stack_sz = 512U, /**< gpio_sweep thread stack size */
00208 } rtos_stack_t;
```

4.7.3.6 timing_t

enum [timing_t](#) : uint32_t

Sweep loop iteration and gap counts.

Enumerator

k_num_pins	Pin count in s_pins
k_delay_iter	nop iterations per HIGH/LOW state (~5 ms at ICLK 96 MHz)
k_gap_iters	delay() calls in the inter-cycle quiet gap

Definition at line 185 of file [main.c](#).

```
00185      : uint32_t {
00186    k_num_pins  = (uint32_t)(sizeof(s_pins) / sizeof(s_pins[0])), /**< Pin count in s_pins */
00187    k_delay_iter = 50000U, /**< nop iterations per HIGH/LOW state (~5 ms at ICLK 96 MHz) */
00188    k_gap_iters  = 500U,  /**< delay() calls in the inter-cycle quiet gap */
00189 } timing_t;
```

4.7.4 Function Documentation

4.7.4.1 clock_init()

```
void clock_init (
    void ) [extern]
```

HOCO 16 MHz -> PLL 192 MHz -> ICLK 96 MHz

Definition at line 100 of file [clock.c](#).

```
00101 {
00102   volatile rx_system_regs_t *sys = system_regs();
00103
00104   /* Step 0: unlock PRC0 (clock) + PRC1 (module stop) */
00105   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00106
00107   /* Step 1: start HOCO if not already running. */
00108   sys->hococr = k_hococr_running;
00109
00110   /* Step 2: wait for HOCO to stabilise. */
00111   while ((sys->oscovfsr & k_oscovfsr_hcovf) == 0U) {
00112     __asm__ volatile("nop");
00113   }
00114 }
```

```

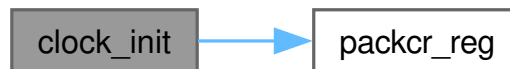
00115  /* Step 3: program PLL: HOCO 16 MHz / 1 * 12 = 192 MHz, and enable. */
00116  sys->pllcr = k_pllcr_hoco_x12;
00117  sys->pllcr2 = k_pllcr2_enable;
00118
00119  /* Step 4: wait for PLL to stabilise. */
00120  while ((sys->oscovfsr & k_oscovfsr_plovf) == 0U) {
00121      __asm__ volatile("nop");
00122  }
00123
00124  /* Step 5: MANDATORY -- raise flash wait state before ICLK exceeds 120 MHz. */
00125  *memwait_reg() = k_memwait_one;
00126
00127  /* Step 6: program all bus dividers. */
00128  sys->sckcr = k_sckcr_value;
00129
00130  /* Step 7: select main PLL as the UCLK divider source (UPLLSEL=0). */
00131  *packcr_reg() = k_packcr_pll_source;
00132
00133  /* Step 8: program the USB clock divider. */
00134  sys->sckcr2 = k_sckcr2_uck_div4;
00135
00136  /* Step 9: switch the system clock source to PLL. */
00137  sys->sckcr3 = k_sckcr3_cksel_pll;
00138
00139  /* Step 10: relock PRCR. */
00140  *prcr_reg() = k_rx_prcr_lock;
00141 }

```

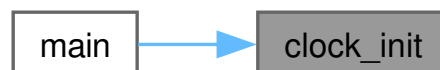
References [k_hococr_running](#), [k_memwait_one](#), [k_oscovfsr_hcovf](#), [k_oscovfsr_plovf](#), [k_packcr_pll_source](#), [k_pllcr2_enable](#), [k_pllcr_hoco_x12](#), [k_sckcr2_uck_div4](#), [k_sckcr_value](#), and [packcr_reg\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.4.2 cmt0_init()

```

void cmt0_init (
    void ) [extern]

```

Program CMT0 for a 100 Hz tick and enable its interrupt.

100 Hz CMT0 tick source for ThreadX

1. Releases CMT0 (and CMT1) from module-stop via MSTPCRA bit 15, bracketed by the PRCR unlock/relock protocol.
2. Stops CMT0, programs CMCR/CMCOR, clears CMCNT.
3. Clears any pending IR, sets IPR=5, enables the vector in IER.
4. Starts CMT0 and globally enables interrupts with SETPSW I.

Precondition

[clock_init\(\)](#) has completed so PCKB = 48 MHz.
 Called from [main\(\)](#) before [tx_kernel_enter\(\)](#).

Postcondition

CMT0 fires at exactly 100 Hz into [cmt0_isr](#).
 Global interrupt flag (PSW.I) is set.

Definition at line 165 of file [cmt0.c](#).

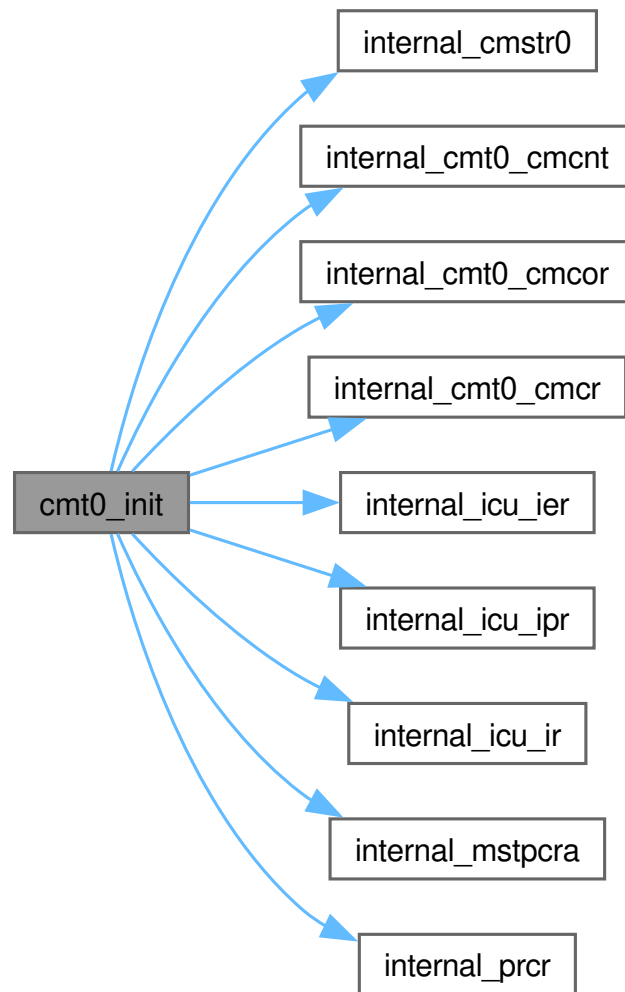
```

00166 {
00167     /* Release CMT0/CMT1 from module stop (MSTPCRA bit 15 = 0) under PRCR protection. */
00168     *internal_prcr() = k_prcr_unlock_prc0_prc1;
00169     *internal_mstpcra() &= ~(uint32_t)(1UL « k_mstpa_cmt0_bit);
00170     *internal_prcr() = k_prcr_lock;
00171
00172     /* Stop CMT0 before reprogramming. */
00173     *internal_cmstr0() &= (uint16_t) ~(uint16_t) k_cmstr0_ch0_bit;
00174
00175     *internal_cmt0_cmcr() = k_cmt0_cmcr_val;
00176     *internal_cmt0_cmcor() = k_cmt0_cmcor_val;
00177     *internal_cmt0_cmcnt() = 0U;
00178
00179     *internal_icu_ir(k_vect_cmt0) = 0U;
00180     *internal_icu_ipr(k_vect_cmt0) = k_cmt0_priority;
00181     *internal_icu_ier(k_icu_ier_cmt0) |= (uint8_t)(1U « k_icu_ier_cmt0_bit);
00182
00183     *internal_cmstr0() |= k_cmstr0_ch0_bit;
00184
00185     /* Inline asm: RX has no standalone GCC builtin for SETPSW; this is the
00186      * idiomatic way to set PSW.I (global interrupt enable) at runtime. */
00187     __asm__ volatile("setpsw i");
00188 }
```

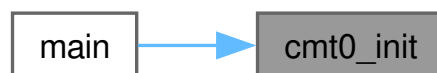
References [internal_cmstr0\(\)](#), [internal_cmt0_cmcnt\(\)](#), [internal_cmt0_cmcor\(\)](#), [internal_cmt0_cmcr\(\)](#), [internal_icu_ier\(\)](#), [internal_icu_ipr\(\)](#), [internal_icu_ir\(\)](#), [internal_mstpcra\(\)](#), [internal_prcr\(\)](#), [k_cmstr0_ch0_bit](#), [k_cmt0_cmcor_val](#), [k_cmt0_cmcr_val](#), [k_cmt0_priority](#), [k_icu_ier_cmt0](#), [k_icu_ier_cmt0_bit](#), [k_mstpa_cmt0_bit](#), [k_prcr_lock](#), [k_prcr_unlock_prc0_prc1](#), and [k_vect_cmt0](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.4.3 internal_delay()

```
void internal_delay (
    void ) [static]
```

Busy-wait for one pin state (~5 ms at ICLK 96 MHz).

Used in place of `tx_thread_sleep`: the 100 Hz ThreadX tick is too coarse for a ~5 ms pulse, and the earlier `tx_thread_sleep`-based version stalled the sweep thread indefinitely (same pattern `blinky_rtos` avoids for its sub-tick timing).

Precondition

ICLK is running at 96 MHz (`clock_init` has completed).

The compiler is honouring volatile on the loop counter.

Postcondition

[k_delay_iter](#) nop instructions have retired and control returns.

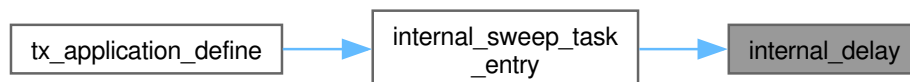
Definition at line 263 of file [main.c](#).

```
00264 {
00265     for (volatile uint32_t i = 0U; i < k_delay_iter; i++) {
00266         /* Inline asm: emit a real NOP so the RX back-end cannot fold the
00267          * loop body away at higher optimisation levels. `volatile i`
00268          * alone is not enough on GCC-RX 14.2 when the body is empty.
00269          * Project coding rule requires a justification at each __asm__ site. */
00270         __asm__ volatile("nop");
00271     }
00272 }
```

References [k_delay_iter](#).

Referenced by [internal_sweep_task_entry\(\)](#).

Here is the caller graph for this function:



4.7.4.4 `internal_gpio_init_all()`

```
void internal_gpio_init_all (
    void ) [static]
```

Configure every pin in [s_pins](#) as a GPIO output driven LOW.

For each entry: clears the PMR bit (GPIO mode, not peripheral), sets the PDR bit (output direction), and clears the PODR bit (LOW). Default RX72N reset state is PMR=0 across the board, so the PMR clear is defensive.

Precondition

[clock_init\(\)](#) has run so PCKB is stable when the writes retire.

[s_pins](#) is a const table in ROM and is safe to read here.

Postcondition

Every pin in [s_pins](#) has PMR=0, PDR=1, PODR=0.

No other port bits were modified (read-modify-write per bit).

Note

Called exactly once, from [tx_application_define](#).

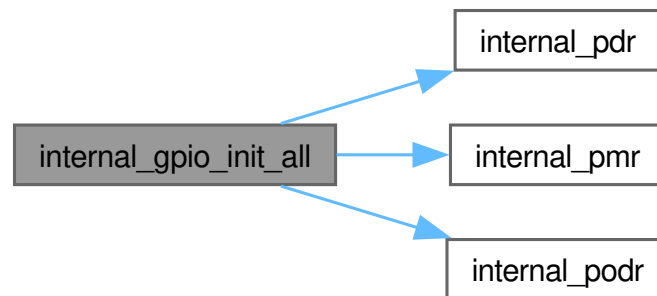
Definition at line 288 of file [main.c](#).

```
00289 {
00290     for (uint32_t i = 0U; i < k_num_pins; i++) {
00291         uint8_t off = (uint8_t)s_pins[i].port_off;
00292         uint8_t mask = (uint8_t)(1U « (uint8_t)s_pins[i].bit);
00293         *internal_pmr(off) &= (uint8_t)~mask;
00294         *internal_pdr(off) |= mask;
00295         *internal_podr(off) &= (uint8_t)~mask;
00296     }
00297 }
```

References [internal_pdr\(\)](#), [internal_pmr\(\)](#), [internal_podr\(\)](#), [k_num_pins](#), and [s_pins](#).

Referenced by [tx_application_define\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.4.5 internal_pdr()

```
volatile uint8_t * internal_pdr (
    uint8_t port_off) [inline], [static]
```

Pointer to the PDR (direction) register for a given port offset.

Parameters

in	port_off	Port offset constant from port_offset_t .
----	----------	---

Returns

Volatile pointer to the 8-bit PDR register for that port.

Definition at line 225 of file [main.c](#).

```
00226 {
00227     return (volatile uint8_t *) (k_pdr_base + port_off);
00228 }
```

References [k_pdr_base](#).

Referenced by [internal_gpio_init_all\(\)](#).

Here is the caller graph for this function:



4.7.4.6 `internal_pmr()`

```
volatile uint8_t * internal_pmr (
    uint8_t port_off)  [inline], [static]
```

Pointer to the PMR (mode) register for a given port offset.

Parameters

in	port_off	Port offset constant from port_offset_t .
----	----------	---

Returns

Volatile pointer to the 8-bit PMR register for that port.

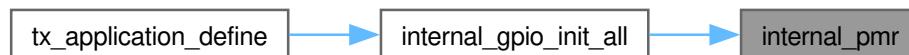
Definition at line 245 of file [main.c](#).

```
00246 {
00247     return (volatile uint8_t *) (k_pmr_base + port_off);
00248 }
```

References [k_pmr_base](#).

Referenced by [internal_gpio_init_all\(\)](#).

Here is the caller graph for this function:



4.7.4.7 `internal_podr()`

```
volatile uint8_t * internal_podr (
    uint8_t port_off)  [inline], [static]
```

Pointer to the PODR (output data) register for a given port offset.

Parameters

in	port_off	Port offset constant from port_offset_t .
----	----------	---

Returns

Volatile pointer to the 8-bit PODR register for that port.

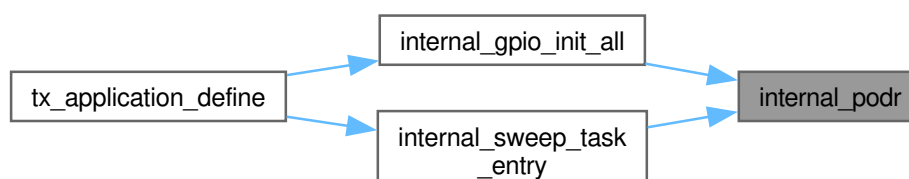
Definition at line 235 of file [main.c](#).

```
00236 {
00237     return (volatile uint8_t *) (k_podr_base + port_off);
00238 }
```

References [k_podr_base](#).

Referenced by [internal_gpio_init_all\(\)](#), and [internal_sweep_task_entry\(\)](#).

Here is the caller graph for this function:



4.7.4.8 internal_sweep_task_entry()

```
void internal_sweep_task_entry (
    ULONG arg) [static]
```

Sweep task: pulses each pin in [s_pins](#) order, forever.

Each iteration drives one pin HIGH, busy-waits, drives it LOW, busy-waits, moves to the next entry. After every pin has pulsed once the thread busy-waits through [k_gap_iters](#) additional delay() calls so the host capture script can anchor on the quiet gap.

Parameters

in	arg	Unused ThreadX thread argument.
----	-----	---------------------------------

Precondition

[internal_gpio_init_all\(\)](#) has completed.

The ThreadX kernel is running (scheduler + tick are up).

Postcondition

Never returns; the loop drives the full [s_pins](#) cycle exactly once per iteration.

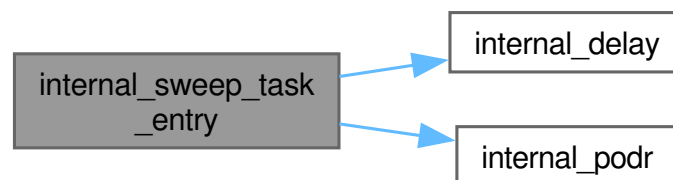
Definition at line 314 of file [main.c](#).

```
00315 {
00316     (void)arg;
00317
00318     for (;;) {
00319         for (uint32_t i = 0U; i < k_num_pins; i++) {
00320             uint8_t off = (uint8_t)s_pins[i].port_off;
00321             uint8_t mask = (uint8_t)(1U « (uint8_t)s_pins[i].bit);
00322             *internal_podr(off) |= mask;
00323             internal_delay();
00324             *internal_podr(off) &= (uint8_t)~mask;
00325             internal_delay();
00326         }
00327         for (uint32_t g = 0U; g < k_gap_iters; g++) {
00328             internal_delay();
00329         }
00330     }
00331 }
```

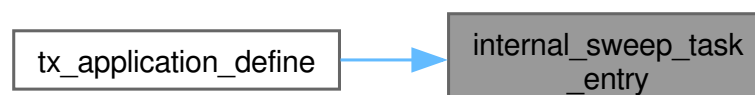
References [internal_delay\(\)](#), [internal_podr\(\)](#), [k_gap_iters](#), [k_num_pins](#), and [s_pins](#).

Referenced by [tx_application_define\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.4.9 main()

```
int main (
    void )
```

Firmware entry point.

Brings up the PLL (96 MHz), arms the CMT0 100 Hz tick, then hands off to ThreadX. Order matters: [clock_init\(\)](#) must run first so PCKB is 48 MHz before [cmt0_init\(\)](#) programs the CMT registers; [cmt0_init\(\)](#) runs before [tx_kernel_enter\(\)](#) as in [blinky_rtos](#).

Returns

Nominally 0, but control never reaches the return: the ThreadX scheduler is the final owner of CPU time.

Return values

0	Unreachable.
---	--------------

Precondition

Called from startup.S with the user/interrupt stack pointers valid and BSS zeroed.

The RX72N reset vector resolves to [__PowerON_Reset_PC](#).

Postcondition

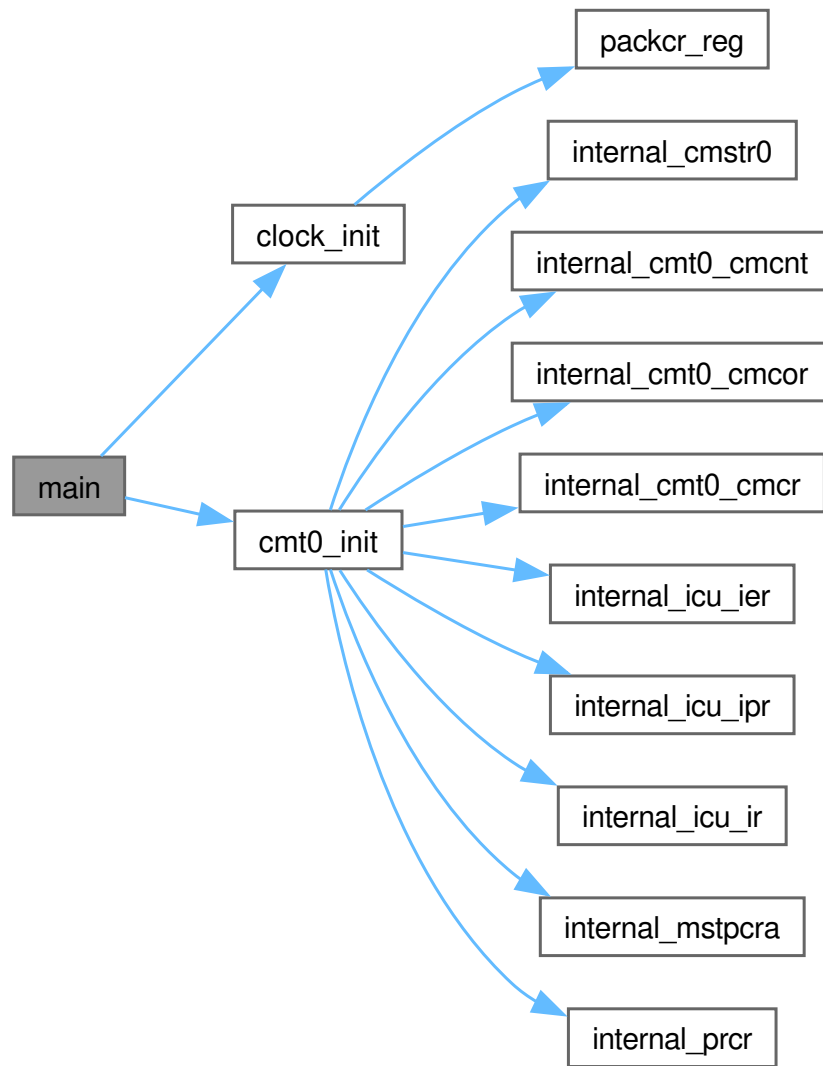
Never returns; ThreadX scheduler owns control.

Definition at line [400](#) of file [main.c](#).

```
00401 {
00402     clock_init();
00403     cmt0_init();
00404     tx_kernel_enter();
00405     return 0;
00406 }
```

References [clock_init\(\)](#), and [cmt0_init\(\)](#).

Here is the call graph for this function:



4.7.4.10 tx_application_define()

```
void tx_application_define (
    void * first_unused_memory)
```

ThreadX hook: configure GPIO and start the sweep task.

ThreadX invokes this from inside `tx_kernel_enter()` once the kernel is initialised. If `tx_thread_create` fails the function parks in an infinite loop so the bench observes a flat line instead of a silently missing sweep (which would otherwise look identical to a crashed thread on the logic analyser).

Parameters

in	first_unused_memory	Unused; kernel-provided heap anchor.
----	---------------------	--------------------------------------

Precondition

The ThreadX kernel has been initialised by `tx_kernel_enter()`.

Postcondition

All `s_pins` are GPIO outputs driven LOW.

Either the `gpio_sweep` thread is created (success) or control is parked in an infinite nop loop (failure).

Definition at line 352 of file `main.c`.

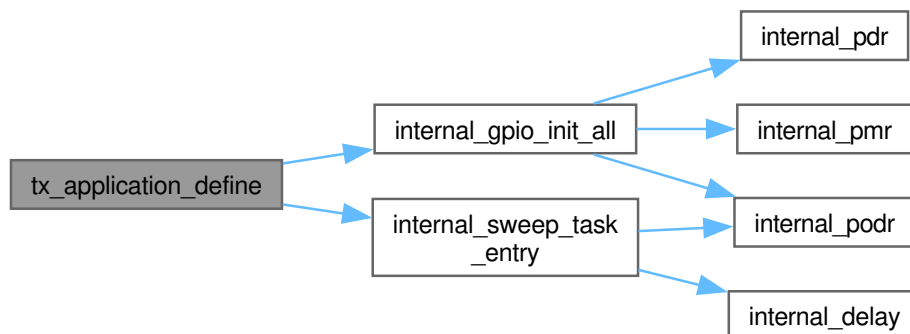
```

00353 {
00354     (void)first_unused_memory;
00355
00356     internal_gpio_init_all();
00357
00358     UINT status = tx_thread_create(&s_sweep_tcb, "gpio_sweep",
00359                                   internal_sweep_task_entry, 0U,
00360                                   s_sweep_stack, k_sweep_stack_sz,
00361                                   k_sweep_priority, k_sweep_priority,
00362                                   TX_NO_TIME_SLICE, TX_AUTO_START);
00363     if (status != TX_SUCCESS) {
00364         /* Park here so the bench observes a flat line instead of a
00365          * ghost sweep. Any failure (TX_SIZE_ERROR / TX_PTR_ERROR /
00366          * TX_START_ERROR) surfaces loudly as "no edges ever" which
00367          * is easy to spot on the logic analyser. */
00368         for (;;) {
00369             __asm__ volatile("nop");
00370         }
00371     }
00372 }

```

References `internal_gpio_init_all()`, `internal_sweep_task_entry()`, `k_sweep_priority`, `k_sweep_stack_sz`, `s_sweep_stack`, and `s_sweep_tcb`.

Here is the call graph for this function:



4.7.5 Variable Documentation

4.7.5.1 s_pins

```
const pin_desc_t s_pins[] [static]
```

Full sweep table, one entry per tested GPIO pin.

Definition at line 135 of file `main.c`.

```

00135     {
00136         { k_port_off_0, k_bit_0 }, { k_port_off_0, k_bit_1 }, { k_port_off_0, k_bit_2 },
00137         { k_port_off_0, k_bit_3 }, { k_port_off_0, k_bit_5 }, { k_port_off_0, k_bit_7 },
00138         { k_port_off_1, k_bit_2 }, { k_port_off_1, k_bit_3 }, { k_port_off_1, k_bit_4 },
00139         { k_port_off_1, k_bit_5 }, { k_port_off_1, k_bit_7 },
00140         { k_port_off_2, k_bit_0 }, { k_port_off_2, k_bit_1 }, { k_port_off_2, k_bit_2 },
00141         { k_port_off_2, k_bit_3 }, { k_port_off_2, k_bit_4 }, { k_port_off_2, k_bit_5 },
00142         { k_port_off_3, k_bit_2 }, { k_port_off_3, k_bit_3 },
00143         { k_port_off_4, k_bit_0 }, { k_port_off_4, k_bit_1 }, { k_port_off_4, k_bit_2 },
00144         { k_port_off_4, k_bit_3 }, { k_port_off_4, k_bit_4 }, { k_port_off_4, k_bit_5 },
00145         { k_port_off_4, k_bit_6 }, { k_port_off_4, k_bit_7 },

```

```

00146 { k_port_off_5, k_bit_0 }, { k_port_off_5, k_bit_1 }, { k_port_off_5, k_bit_2 },
00147 { k_port_off_5, k_bit_3 }, { k_port_off_5, k_bit_4 }, { k_port_off_5, k_bit_5 },
00148 { k_port_off_5, k_bit_6 },
00149 { k_port_off_6, k_bit_0 }, { k_port_off_6, k_bit_1 }, { k_port_off_6, k_bit_2 },
00150 { k_port_off_6, k_bit_3 }, { k_port_off_6, k_bit_4 }, { k_port_off_6, k_bit_5 },
00151 { k_port_off_6, k_bit_6 }, { k_port_off_6, k_bit_7 },
00152 { k_port_off_7, k_bit_0 }, { k_port_off_7, k_bit_1 }, { k_port_off_7, k_bit_2 },
00153 { k_port_off_7, k_bit_3 }, { k_port_off_7, k_bit_4 }, { k_port_off_7, k_bit_5 },
00154 { k_port_off_7, k_bit_6 }, { k_port_off_7, k_bit_7 },
00155 { k_port_off_8, k_bit_0 }, { k_port_off_8, k_bit_1 }, { k_port_off_8, k_bit_2 },
00156 { k_port_off_8, k_bit_3 }, { k_port_off_8, k_bit_6 }, { k_port_off_8, k_bit_7 },
00157 { k_port_off_9, k_bit_0 }, { k_port_off_9, k_bit_1 }, { k_port_off_9, k_bit_2 },
00158 { k_port_off_9, k_bit_3 },
00159 { k_port_off_a, k_bit_0 }, { k_port_off_a, k_bit_1 }, { k_port_off_a, k_bit_2 },
00160 { k_port_off_a, k_bit_3 }, { k_port_off_a, k_bit_4 }, { k_port_off_a, k_bit_5 },
00161 { k_port_off_a, k_bit_6 }, { k_port_off_a, k_bit_7 },
00162 { k_port_off_b, k_bit_0 }, { k_port_off_b, k_bit_1 }, { k_port_off_b, k_bit_2 },
00163 { k_port_off_b, k_bit_3 }, { k_port_off_b, k_bit_4 }, { k_port_off_b, k_bit_5 },
00164 { k_port_off_c, k_bit_0 }, { k_port_off_c, k_bit_1 }, { k_port_off_c, k_bit_2 },
00165 { k_port_off_c, k_bit_3 }, { k_port_off_c, k_bit_4 }, { k_port_off_c, k_bit_5 },
00166 { k_port_off_c, k_bit_6 },
00167 { k_port_off_d, k_bit_0 }, { k_port_off_d, k_bit_1 }, { k_port_off_d, k_bit_2 },
00168 { k_port_off_d, k_bit_3 }, { k_port_off_d, k_bit_4 }, { k_port_off_d, k_bit_5 },
00169 { k_port_off_d, k_bit_6 }, { k_port_off_d, k_bit_7 },
00170 { k_port_off_e, k_bit_0 }, { k_port_off_e, k_bit_1 }, { k_port_off_e, k_bit_2 },
00171 { k_port_off_e, k_bit_3 }, { k_port_off_e, k_bit_4 }, { k_port_off_e, k_bit_5 },
00172 { k_port_off_e, k_bit_6 }, { k_port_off_e, k_bit_7 },
00173 { k_port_off_f, k_bit_5 },
00174 };

```

Referenced by [internal_gpio_init_all\(\)](#), and [internal_sweep_task_entry\(\)](#).

4.7.5.2 s_sweep_stack

```
uint8_t s_sweep_stack[k_sweep_stack_sz] [static]
```

Definition at line 214 of file [main.c](#).

Referenced by [tx_application_define\(\)](#).

4.7.5.3 s_sweep_tcb

```
TX_THREAD s_sweep_tcb [static]
```

Definition at line 211 of file [main.c](#).

Referenced by [tx_application_define\(\)](#).

4.8 main.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file main.c
00003  * @brief GPIO breakout verification firmware for RX72N, ThreadX edition.
00004  *
00005  * @details
00006  * Mirrors the structure that works in blinky_rtos: one ThreadX thread that
00007  * drives the pins with a busy-wait delay (NOT tx_thread_sleep, which stalls
00008  * when the scheduler/tick isn't ticking for any reason). A single 100 Hz
00009  * CMT0 tick keeps the kernel happy even though we do not use it for
00010  * sleep-based timing.
00011  *
00012  * Test pattern per pin:
00013  * 1. Drive HIGH for ~5 ms (busy-wait loop)
00014  * 2. Drive LOW for ~5 ms
00015  * 3. Advance to next pin
00016  *

```

```

00017 * After all pins are tested a long quiet gap (~2.3 s) separates cycles so
00018 * the host script can anchor capture timing to the gap.
00019 *
00020 * Clock path:
00021 * clock_init() brings us from HOCO 16 MHz -> PLL 192 MHz -> ICLK 96 MHz
00022 * / PCKB 48 MHz. CMT0 is then programmed at PCKB/32 for 100 Hz tick.
00023 *
00024 * PJ3 and PJ5 are excluded: on RX72N they double as JTAG TMS/TDO and driving
00025 * them while the E2 Lite has the debug interface latched hangs the MCU.
00026 *
00027 * SPDX-License-Identifier: MIT
00028 * @copyright Copyright (c) 2026 Locked Inc.
00029 */
00030
00031 #include <stdint.h>
00032 #include "tx_api.h"
00033
00034 /*
=====
00035 * GPIO register base addresses (RX72N Hardware Manual Ch 22)
00036 *
=====
*/
00037 /**
00038 * @enum port_reg_base_t
00039 * @brief Base addresses of the per-port GPIO register banks.
00040 *
00041 * @details
00042 * Each GPIO port has three 8-bit registers indexed by `port_offset_t`:
00043 * - PDR (direction): 0 = input, 1 = output
00044 * - PODR (output data): 0 = LOW, 1 = HIGH
00045 * - PMR (mode): 0 = GPIO, 1 = peripheral
00046 * Address = base + port_offset.
00047 *
00048 * @see RX72N Hardware Manual, Chapter 22 (I/O Ports)
00049 */
00050 typedef enum : uintptr_t {
00051     k_pdr_base = 0x0008C000U, /**< Port direction register bank base */
00052     k_podr_base = 0x0008C020U, /**< Port output data register bank base */
00053     k_pmr_base = 0x0008C060U, /**< Port mode register bank base */
00054 } port_reg_base_t;
00055
00056 /*
=====
00057 * Port offsets (PDR[k_port_off_x] = PDR_base + offset)
00058 *
=====
*/
00059 /**
00060 * @enum port_offset_t
00061 * @brief Per-port index added to every GPIO register bank base.
00062 *
00063 * @details
00064 * Ports 0..9 map to offsets 0x00..0x09, ports A..F to 0x0A..0x0F. Port J
00065 * lives at offset 0x12 because offsets 0x10 (Port G) and 0x11 (Port H) are
00066 * reserved on this package even though the address space is allocated.
00067 */
00068 typedef enum : uint8_t {
00069     k_port_off_0 = 0x00U, /**< Port 0 offset within each register bank */
00070     k_port_off_1 = 0x01U, /**< Port 1 offset within each register bank */
00071     k_port_off_2 = 0x02U, /**< Port 2 offset within each register bank */
00072     k_port_off_3 = 0x03U, /**< Port 3 offset within each register bank */
00073     k_port_off_4 = 0x04U, /**< Port 4 offset within each register bank */
00074     k_port_off_5 = 0x05U, /**< Port 5 offset within each register bank */
00075     k_port_off_6 = 0x06U, /**< Port 6 offset within each register bank */
00076     k_port_off_7 = 0x07U, /**< Port 7 offset within each register bank */
00077     k_port_off_8 = 0x08U, /**< Port 8 offset within each register bank */
00078     k_port_off_9 = 0x09U, /**< Port 9 offset within each register bank */
00079     k_port_off_a = 0x0AU, /**< Port A offset within each register bank */
00080     k_port_off_b = 0x0BU, /**< Port B offset within each register bank */
00081     k_port_off_c = 0x0CU, /**< Port C offset within each register bank */
00082     k_port_off_d = 0x0DU, /**< Port D offset within each register bank */
00083     k_port_off_e = 0x0EU, /**< Port E offset within each register bank */
00084     k_port_off_f = 0x0FU, /**< Port F offset within each register bank */
00085 } port_offset_t;
00086
00087 /*
=====
00088 * Port bit positions (0..7 within each 8-bit PDR/PODR/PMR register)
00089 *
=====
*/
00090 /**
00091 * @enum port_bit_t
00092 * @brief Bit position within an 8-bit port register.
00093 *
00094 * @details

```



```

00095 * RX72N ports are 8 bits wide; pin N of a given port is bit N of the port's
00096 * PDR/PODR/PMR. Using a typed enum in `s_pins[]` keeps the table readable
00097 * and satisfies the STAR coding rule against raw numeric literals.
00098 */
00099 typedef enum : uint8_t {
00100     k_bit_0 = 0U, /**< Bit 0 (pin 0) of the port */
00101     k_bit_1 = 1U, /**< Bit 1 (pin 1) of the port */
00102     k_bit_2 = 2U, /**< Bit 2 (pin 2) of the port */
00103     k_bit_3 = 3U, /**< Bit 3 (pin 3) of the port */
00104     k_bit_4 = 4U, /**< Bit 4 (pin 4) of the port */
00105     k_bit_5 = 5U, /**< Bit 5 (pin 5) of the port */
00106     k_bit_6 = 6U, /**< Bit 6 (pin 6) of the port */
00107     k_bit_7 = 7U, /**< Bit 7 (pin 7) of the port */
00108 } port_bit_t;
00109
00110 /*
=====
00111 * Pin descriptor
00112 *
=====
*/
00113 /**
00114 * @struct pin_desc_t
00115 * @brief One entry in the firmware's pin sweep table.
00116 *
00117 * @details
00118 * The sweep iterates `s_pins[]` in order; its index is the firmware pin
00119 * index matched by the host capture script.
00120 */
00121 typedef struct {
00122     port_offset_t port_off; /**< Port offset, used with k_pdr_base/k_podr_base/k_pmr_base */
00123     port_bit_t bit; /**< Bit position (0..7) within the 8-bit port register */
00124 } pin_desc_t;
00125
00126 /*
=====
00127 * Complete pin table. Array index = firmware sweep index. BGA pin numbers
00128 * from `pin_map.md` are deliberately NOT encoded here -- the firmware only
00129 * cares about port/bit. PJ3 and PJ5 are intentionally omitted: on RX72N
00130 * they double as JTAG TMS/TDO; driving them while the E2 Lite holds the
00131 * debug interface hangs the MCU (empirically observed, every pin after
00132 * the first PJ write went silent).
00133 *
=====
*/
00134 /** @brief Full sweep table, one entry per tested GPIO pin. */
00135 static const pin_desc_t s_pins[] = {
00136     { k_port_off_0, k_bit_0 }, { k_port_off_0, k_bit_1 }, { k_port_off_0, k_bit_2 },
00137     { k_port_off_0, k_bit_3 }, { k_port_off_0, k_bit_5 }, { k_port_off_0, k_bit_7 },
00138     { k_port_off_1, k_bit_2 }, { k_port_off_1, k_bit_3 }, { k_port_off_1, k_bit_4 },
00139     { k_port_off_1, k_bit_5 }, { k_port_off_1, k_bit_7 },
00140     { k_port_off_2, k_bit_0 }, { k_port_off_2, k_bit_1 }, { k_port_off_2, k_bit_2 },
00141     { k_port_off_2, k_bit_3 }, { k_port_off_2, k_bit_4 }, { k_port_off_2, k_bit_5 },
00142     { k_port_off_3, k_bit_2 }, { k_port_off_3, k_bit_3 },
00143     { k_port_off_4, k_bit_0 }, { k_port_off_4, k_bit_1 }, { k_port_off_4, k_bit_2 },
00144     { k_port_off_4, k_bit_3 }, { k_port_off_4, k_bit_4 }, { k_port_off_4, k_bit_5 },
00145     { k_port_off_4, k_bit_6 }, { k_port_off_4, k_bit_7 },
00146     { k_port_off_5, k_bit_0 }, { k_port_off_5, k_bit_1 }, { k_port_off_5, k_bit_2 },
00147     { k_port_off_5, k_bit_3 }, { k_port_off_5, k_bit_4 }, { k_port_off_5, k_bit_5 },
00148     { k_port_off_5, k_bit_6 },
00149     { k_port_off_6, k_bit_0 }, { k_port_off_6, k_bit_1 }, { k_port_off_6, k_bit_2 },
00150     { k_port_off_6, k_bit_3 }, { k_port_off_6, k_bit_4 }, { k_port_off_6, k_bit_5 },
00151     { k_port_off_6, k_bit_6 }, { k_port_off_6, k_bit_7 },
00152     { k_port_off_7, k_bit_0 }, { k_port_off_7, k_bit_1 }, { k_port_off_7, k_bit_2 },
00153     { k_port_off_7, k_bit_3 }, { k_port_off_7, k_bit_4 }, { k_port_off_7, k_bit_5 },
00154     { k_port_off_7, k_bit_6 }, { k_port_off_7, k_bit_7 },
00155     { k_port_off_8, k_bit_0 }, { k_port_off_8, k_bit_1 }, { k_port_off_8, k_bit_2 },
00156     { k_port_off_8, k_bit_3 }, { k_port_off_8, k_bit_6 }, { k_port_off_8, k_bit_7 },
00157     { k_port_off_9, k_bit_0 }, { k_port_off_9, k_bit_1 }, { k_port_off_9, k_bit_2 },
00158     { k_port_off_9, k_bit_3 },
00159     { k_port_off_a, k_bit_0 }, { k_port_off_a, k_bit_1 }, { k_port_off_a, k_bit_2 },
00160     { k_port_off_a, k_bit_3 }, { k_port_off_a, k_bit_4 }, { k_port_off_a, k_bit_5 },
00161     { k_port_off_a, k_bit_6 }, { k_port_off_a, k_bit_7 },
00162     { k_port_off_b, k_bit_0 }, { k_port_off_b, k_bit_1 }, { k_port_off_b, k_bit_2 },
00163     { k_port_off_b, k_bit_3 }, { k_port_off_b, k_bit_4 }, { k_port_off_b, k_bit_5 },
00164     { k_port_off_c, k_bit_0 }, { k_port_off_c, k_bit_1 }, { k_port_off_c, k_bit_2 },
00165     { k_port_off_c, k_bit_3 }, { k_port_off_c, k_bit_4 }, { k_port_off_c, k_bit_5 },
00166     { k_port_off_c, k_bit_6 },
00167     { k_port_off_d, k_bit_0 }, { k_port_off_d, k_bit_1 }, { k_port_off_d, k_bit_2 },
00168     { k_port_off_d, k_bit_3 }, { k_port_off_d, k_bit_4 }, { k_port_off_d, k_bit_5 },
00169     { k_port_off_d, k_bit_6 }, { k_port_off_d, k_bit_7 },
00170     { k_port_off_e, k_bit_0 }, { k_port_off_e, k_bit_1 }, { k_port_off_e, k_bit_2 },
00171     { k_port_off_e, k_bit_3 }, { k_port_off_e, k_bit_4 }, { k_port_off_e, k_bit_5 },
00172     { k_port_off_e, k_bit_6 }, { k_port_off_e, k_bit_7 },
00173     { k_port_off_f, k_bit_5 },
00174 };
00175

```

```

00176 /*
=====
00177 * Timing constants. Busy-wait delay at ICLK = 96 MHz. The actual pin
00178 * period drifts a little with optimisation level; gpio_verify.py measures
00179 * it empirically from the capture.
00180 *
=====
*/
00181 /**
00182 * @enum timing_t
00183 * @brief Sweep loop iteration and gap counts.
00184 */
00185 typedef enum : uint32_t {
00186     k_num_pins    = (uint32_t)(sizeof(s_pins) / sizeof(s_pins[0])), /**< Pin count in s_pins */
00187     k_delay_iter  = 50000U, /**< nop iterations per HIGH/LOW state (~5 ms at ICLK 96 MHz) */
00188     k_gap_iters   = 500U, /**< delay() calls in the inter-cycle quiet gap */
00189 } timing_t;
00190
00191 /*
=====
00192 * ThreadX objects
00193 *
=====
*/
00194 /**
00195 * @enum rtos_prio_t
00196 * @brief ThreadX priority numbers used by this firmware.
00197 */
00198 typedef enum : uint8_t {
00199     k_sweep_priority = 10U, /**< Priority of the gpio_sweep thread */
00200 } rtos_prio_t;
00201
00202 /**
00203 * @enum rtos_stack_t
00204 * @brief ThreadX stack sizes in bytes.
00205 */
00206 typedef enum : uint16_t {
00207     k_sweep_stack_sz = 512U, /**< gpio_sweep thread stack size */
00208 } rtos_stack_t;
00209
00210 /** @var s_sweep_tcb @brief ThreadX control block for the gpio_sweep task. */
00211 static TX_THREAD s_sweep_tcb;
00212
00213 /** @var s_sweep_stack @brief Static stack backing s_sweep_tcb. */
00214 static uint8_t s_sweep_stack[k_sweep_stack_sz];
00215
00216 /*
=====
00217 * Inline accessors
00218 *
=====
*/
00219
00220 /**
00221 * @brief Pointer to the PDR (direction) register for a given port offset.
00222 * @param[in] port_off Port offset constant from `port_offset_t`.
00223 * @return Volatile pointer to the 8-bit PDR register for that port.
00224 */
00225 static inline volatile uint8_t *internal_pdr(uint8_t port_off)
00226 {
00227     return (volatile uint8_t *) (k_pdr_base + port_off);
00228 }
00229
00230 /**
00231 * @brief Pointer to the PODR (output data) register for a given port offset.
00232 * @param[in] port_off Port offset constant from `port_offset_t`.
00233 * @return Volatile pointer to the 8-bit PODR register for that port.
00234 */
00235 static inline volatile uint8_t *internal_podr(uint8_t port_off)
00236 {
00237     return (volatile uint8_t *) (k_podr_base + port_off);
00238 }
00239
00240 /**
00241 * @brief Pointer to the PMR (mode) register for a given port offset.
00242 * @param[in] port_off Port offset constant from `port_offset_t`.
00243 * @return Volatile pointer to the 8-bit PMR register for that port.
00244 */
00245 static inline volatile uint8_t *internal_pmr(uint8_t port_off)
00246 {
00247     return (volatile uint8_t *) (k_pmr_base + port_off);
00248 }
00249
00250 /**
00251 * @brief Busy-wait for one pin state (~5 ms at ICLK 96 MHz).
00252 *
00253 * @details

```

```

00254 * Used in place of `tx_thread_sleep`: the 100 Hz ThreadX tick is too coarse
00255 * for a ~5 ms pulse, and the earlier `tx_thread_sleep`-based version stalled
00256 * the sweep thread indefinitely (same pattern blinky_rtos avoids for its
00257 * sub-tick timing).
00258 *
00259 * @pre ICLK is running at 96 MHz (clock_init has completed).
00260 * @pre The compiler is honouring `volatile` on the loop counter.
00261 * @post `k_delay_iter` nop instructions have retired and control returns.
00262 */
00263 static void internal_delay(void)
00264 {
00265     for (volatile uint32_t i = 0U; i < k_delay_iter; i++) {
00266         /* Inline asm: emit a real NOP so the RX back-end cannot fold the
00267          * loop body away at higher optimisation levels. `volatile i`
00268          * alone is not enough on GCC-RX 14.2 when the body is empty.
00269          * Project coding rule requires a justification at each __asm__ site. */
00270         __asm__ volatile("nop");
00271     }
00272 }
00273
00274 /**
00275  * @brief Configure every pin in `s_pins` as a GPIO output driven LOW.
00276  *
00277  * @details
00278  * For each entry: clears the PMR bit (GPIO mode, not peripheral), sets the
00279  * PDR bit (output direction), and clears the PODR bit (LOW). Default RX72N
00280  * reset state is PMR=0 across the board, so the PMR clear is defensive.
00281  *
00282  * @pre `clock_init()` has run so PCKB is stable when the writes retire.
00283  * @pre `s_pins` is a `const` table in ROM and is safe to read here.
00284  * @post Every pin in `s_pins` has PMR=0, PDR=1, PODR=0.
00285  * @post No other port bits were modified (read-modify-write per bit).
00286  * @note Called exactly once, from `tx_application_define`.
00287 */
00288 static void internal_gpio_init_all(void)
00289 {
00290     for (uint32_t i = 0U; i < k_num_pins; i++) {
00291         uint8_t off = (uint8_t)s_pins[i].port_off;
00292         uint8_t mask = (uint8_t)(1U « (uint8_t)s_pins[i].bit);
00293         *internal_pmr(off) &= (uint8_t)~mask;
00294         *internal_pdr(off) |= mask;
00295         *internal_podr(off) &= (uint8_t)~mask;
00296     }
00297 }
00298
00299 /**
00300  * @brief Sweep task: pulses each pin in `s_pins` order, forever.
00301  *
00302  * @details
00303  * Each iteration drives one pin HIGH, busy-waits, drives it LOW,
00304  * busy-waits, moves to the next entry. After every pin has pulsed once
00305  * the thread busy-waits through `k_gap_iters` additional `delay()` calls
00306  * so the host capture script can anchor on the quiet gap.
00307  *
00308  * @param[in] arg Unused ThreadX thread argument.
00309  * @pre `internal_gpio_init_all()` has completed.
00310  * @pre The ThreadX kernel is running (scheduler + tick are up).
00311  * @post Never returns; the loop drives the full `s_pins` cycle exactly
00312  *         once per iteration.
00313 */
00314 static void internal_sweep_task_entry(ULONG arg)
00315 {
00316     (void)arg;
00317
00318     for (;;) {
00319         for (uint32_t i = 0U; i < k_num_pins; i++) {
00320             uint8_t off = (uint8_t)s_pins[i].port_off;
00321             uint8_t mask = (uint8_t)(1U « (uint8_t)s_pins[i].bit);
00322             *internal_podr(off) |= mask;
00323             internal_delay();
00324             *internal_podr(off) &= (uint8_t)~mask;
00325             internal_delay();
00326         }
00327         for (uint32_t g = 0U; g < k_gap_iters; g++) {
00328             internal_delay();
00329         }
00330     }
00331 }
00332
00333 /**
00334  * tx_application_define -- called by ThreadX from tx_kernel_enter().
00335  *
00336  * @brief ThreadX hook: configure GPIO and start the sweep task.

```

```

00338 *
00339 * @details
00340 * ThreadX invokes this from inside `tx_kernel_enter()` once the kernel
00341 * is initialised. If `tx_thread_create` fails the function parks in an
00342 * infinite loop so the bench observes a flat line instead of a silently
00343 * missing sweep (which would otherwise look identical to a crashed
00344 * thread on the logic analyser).
00345 *
00346 * @param[in] first_unused_memory Unused; kernel-provided heap anchor.
00347 * @pre The ThreadX kernel has been initialised by `tx_kernel_enter()`.
00348 * @post All `s_pins` are GPIO outputs driven LOW.
00349 * @post Either the gpio_sweep thread is created (success) or control is
00350 * parked in an infinite nop loop (failure).
00351 */
00352 void tx_application_define(void *first_unused_memory)
00353 {
00354     (void)first_unused_memory;
00355
00356     internal_gpio_init_all();
00357
00358     UINT status = tx_thread_create(&s_sweep_tcb, "gpio_sweep",
00359                                     internal_sweep_task_entry, 0U,
00360                                     s_sweep_stack, k_sweep_stack_sz,
00361                                     k_sweep_priority, k_sweep_priority,
00362                                     TX_NO_TIME_SLICE, TX_AUTO_START);
00363     if (status != TX_SUCCESS) {
00364         /* Park here so the bench observes a flat line instead of a
00365          * ghost sweep. Any failure (TX_SIZE_ERROR / TX_PTR_ERROR /
00366          * TX_START_ERROR) surfaces loudly as "no edges ever" which
00367          * is easy to spot on the logic analyser. */
00368         for (;;) {
00369             __asm__ volatile("nop");
00370         }
00371     }
00372 }
00373
00374 /*
=====
00375 * External init hooks
00376 *
=====
00377 extern void clock_init(void); /**< HOCO 16 MHz -> PLL 192 MHz -> ICLK 96 MHz */
00378 extern void cmt0_init(void); /**< 100 Hz CMT0 tick source for ThreadX */
00379
00380 /*
=====
00381 * main
00382 *
=====
00383 /**
00384 * @brief Firmware entry point.
00385 *
00386 * @details
00387 * Brings up the PLL (96 MHz), arms the CMT0 100 Hz tick, then hands off
00388 * to ThreadX. Order matters: `clock_init()` must run first so PCKB is
00389 * 48 MHz before `cmt0_init()` programs the CMT registers; `cmt0_init()`
00390 * runs before `tx_kernel_enter()` as in blinky_rtos.
00391 *
00392 * @return Nominally 0, but control never reaches the `return`: the
00393 * ThreadX scheduler is the final owner of CPU time.
00394 * @retval 0 Unreachable.
00395 * @pre Called from `startup.S` with the user/interrupt stack pointers
00396 * valid and BSS zeroed.
00397 * @pre The RX72N reset vector resolves to `_PowerON_Reset_PC`.
00398 * @post Never returns; ThreadX scheduler owns control.
00399 */
00400 int main(void)
00401 {
00402     clock_init();
00403     cmt0_init();
00404     tx_kernel_enter();
00405     return 0;
00406 }

```